M03 — Blazor Web App y componentes Razor

Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

CURSO: Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: TEORÍA

AUTOR: JAIME GALLO

Propósito del módulo

En los dos primeros módulos construimos una aplicación MVC que recibe HTTP, selecciona acciones de controlador, genera HTML a partir de vistas Razor y, finalmente, guarda productos con EF Core. También comprobamos que Razor Pages permite organizar una pequeña pantalla alrededor de una clase PageModel. En este tercer módulo construiremos **la aplicación B**, una Blazor Web App de .NET 8, para aprender otro modelo de programación de la interfaz: una pantalla puede estar formada por componentes que mantienen estado y responden a eventos sin repetir el ciclo completo de petición y documento HTML por cada interacción.

No trasladaremos el CRUD MVC a Blazor ni añadiremos otra base de datos. Tampoco consumiremos todavía una API REST: eso pertenece a M04. El dominio sigue siendo un pequeño catálogo de productos para que la nueva dificultad resida en los componentes, no en el negocio. La distinción importa: la existencia de dos aplicaciones docentes no significa que todas las técnicas deban mezclarse en un mismo proyecto.

Este manual desarrolla los puntos 3.1–3.6. La práctica correspondiente contiene instrucciones, rutas, bloques de código preparados, resultados positivos y negativos, ejercicios autónomos, retos y soluciones. El itinerario de seis horas tiene un recorrido esencial; las profundizaciones que siguen ofrecen un manual de consulta y no multiplican las tareas obligatorias de clase.

Índice del módulo

- 3.1. Blazor frente a MVC, SSR estático e Interactive Server.
- 3.2. Estructura de Blazor Web App, componentes, páginas y navegación.
- 3.3. Parámetros, reutilización y composición de componentes.
- 3.4. Eventos, @onclick, @bind y estado local.
- 3.5. Comunicación padre–hijo y EventCallback.
- 3.6. OnInitializedAsync, inyección y ciclo de vida esencial.

3.1. Blazor Web App: de una respuesta HTML a una interfaz que conserva estado

El problema que queremos resolver

Hasta ahora sabemos conseguir que el navegador muestre un catálogo y permita modificarlo. Con MVC podemos construir casi cualquier interfaz web, pero una interacción tradicional consiste con frecuencia en enviar HTTP, ejecutar un controlador y devolver una vista o una redirección. Si queremos que un pequeño botón incremente un contador, podemos usar JavaScript para gestionar ese estado en el navegador o realizar una nueva solicitud al servidor. Blazor proporciona otra posibilidad: describir la interfaz con componentes Razor en C# y conectar los eventos de interfaz con métodos del componente.

No hay que confundir **Blazor** con «un MVC al que hemos añadido etiquetas nuevas». Cambia el lugar donde se expresa la interacción y cómo se mantiene el estado de la interfaz. Tampoco es correcto afirmar que Blazor elimina HTTP. La primera navegación obtiene un documento y los recursos de la aplicación por HTTP. Con Interactive Server, los eventos posteriores circulan entre el navegador y el servidor mediante una conexión interactiva; el framework comunica los cambios de representación necesarios.

Nuestro primer experimento compara dos páginas del **mismo host**. La página `/estatica` muestra un botón cuya expresión Razor contiene `@onclick`, pero no declara un modo interactivo. La página `/interactiva` presenta un contador semejante y declara `@rendermode InteractiveServer`. En el primer caso podemos pulsar sin que el método C# se ejecute en respuesta al clic; en el segundo, la cifra cambia. La diferencia no se explica por el nombre del botón ni por el lenguaje empleado, sino por la capacidad real de procesar eventos.

Tres tecnologías que comparten Razor, pero no el mismo ciclo

Una **vista Razor MVC** es una plantilla `.cshtml` que el controlador selecciona para generar HTML. El modelo tipado suele representar los datos de una solicitud y la vista no se convierte por sí misma en un endpoint navegable.

Una **Razor Page** también usa `.cshtml`, pero contiene `@page` y suele asociarse a un `PageModel` con handlers como `OnGet` y `OnPost`. Un **componente Razor de Blazor** utiliza habitualmente la extensión `.razor`; puede recibir parámetros, representar elementos reutilizables, mantener campos y manejar eventos cuando se ejecuta bajo un modo interactivo.

Las tres tecnologías comparten C#, HTML y sintaxis Razor, pero no deberíamos deducir equivalencia por la presencia de un símbolo `@`. En MVC, una expresión `@Model.Nombre` lee el modelo enviado a la vista; en Razor Pages, `@Model.Mensaje` consulta la instancia de PageModel; en un componente Blazor, `@contador` puede leer un campo del propio componente, cuyo valor se modifica a través de `@onclick` durante una sesión interactiva.

Esta diferencia evita uno de los errores más habituales del curso: trasladar una acción de controlador a un componente porque ambas «tienen C#», sin comprender cuándo se invoca cada pieza. La acción suele ser seleccionada como endpoint HTTP. Un manejador de eventos de Blazor puede ejecutarse después de que la página esté representada, debido a una interacción gestionada por el framework.

Renderizado estático en el servidor

El término **SSR** significa *server-side rendering*: el servidor genera una representación inicial que recibe el navegador. En el SSR estático, la petición produce HTML y termina; esa representación no establece por sí sola un circuito interactivo de Blazor para ejecutar automáticamente métodos de C# después de un clic.

Nuestro archivo `Components/Pages/Estatica.razor` contiene, entre otros elementos:

```
@page "/estatica"
<h1>P3.1 | Componente con SSR estático</h1>
<p id="contador-estatico">Pulsaciones: @contador</p>
<button id="boton-estatico" @onclick="Incrementar">Pulsar (sin interacción)</
button>

@code {
    private int contador;
```

```
    private void Incrementar() => contador++;
}
```

Aquí el componente **sí contiene un método**. Su código es válido y compila, pero el mero atributo `@onclick` no convierte el endpoint en interactivo. Al cargar la página vemos `Pulsaciones: 0`; pulsar no cambia ese texto. Esta observación constituye una prueba conceptual: no basta con que exista código de evento en el proyecto; debemos tener el renderizado y los servicios apropiados.

SSR estático no equivale a «páginas sin C#». El servidor puede construir datos, aplicar lógica al renderizar y mostrar contenido dinámico entre distintas solicitudes. La limitación concreta que observamos es la ausencia de una **sesión interactiva de componente** para responder a ese clic.

Interactive Server en .NET 8

La página `Components/Pages/Interactiva.razor` define una ruta y un modo explícito:

```
@page "/interactiva"
@rendermode InteractiveServer

<h1>P3.1 | Componente interactivo en servidor</h1>
<p id="contador-interactivo">Pulsaciones: @contador</p>
<button id="boton-interactivo" @onclick="Incrementar">Pulsar (interactivo)</
button>

@code {
    private int contador;
    private void Incrementar() => contador++;
}
```

La diferencia visible es `@rendermode InteractiveServer`. Ahora, tras establecerse la conexión interactiva, el clic invoca `Incrementar`, modifica el campo `contador` y el framework actualiza la representación visible. No se necesita escribir una acción HTTP GET o POST para incrementar el campo ni programar manualmente una modificación del DOM con JavaScript.

Una precisión importante: el estado `contador` vive en el componente asociado a esa sesión interactiva; no es una variable global de todos los usuarios, una columna SQL ni una propiedad persistente. Si el circuito se pierde o se crea

una nueva instancia, el valor puede empezar de nuevo. Más adelante usaremos una colección de productos en memoria, con la misma limitación.

Qué registra realmente Program.cs

En los seis estados del proyecto, el host utiliza estos elementos esenciales:

```
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

app.MapRazorComponents<App>()
   .AddInteractiveServerRenderMode();
```

La primera parte incorpora los servicios que permiten construir componentes Razor y atender interactividad en el servidor. La segunda mapea los endpoints y habilita el modo Interactive Server. La aplicación incluye además `app.UseStaticFiles()` y `app.UseAntiforgery()` para los recursos y la protección correspondiente.

Este ejemplo escoge **interactividad por página**, no global. El archivo `Components/App.razor` contiene `<Routes />` sin aplicar un modo interactivo a todo el árbol; cada página que necesita interacción declara `@rendermode InteractiveServer`. Así podemos comparar `/estatica` y `/interactiva` bajo el mismo host. Otra configuración válida sería establecer el modo a nivel de `Routes`; no la utilizamos aquí porque haría menos clara la comparación con SSR estático.

El script `_framework/blazor.web.js`, incluido al final de `App.razor`, permite que la aplicación configure la experiencia de navegación e interactividad en el cliente. No sustituye a los servicios y endpoints registrados en `Program.cs`. Los tres lugares —registro de servicios, mapeo del servidor y modo de renderizado— cumplen responsabilidades relacionadas pero distintas.

Qué ocurre después del clic

El recorrido puede describirse en pasos causales. El navegador recibe primero la página y sus recursos; el runtime de Blazor establece la conexión necesaria con el servidor para el componente interactivo. Cuando pulsamos, el evento se identifica en el navegador y el framework lo comunica al circuito. El

método `Incrementar` de la instancia de componente se ejecuta en el servidor. Cambia `contador` y Blazor calcula y transmite la actualización de la interfaz. El navegador representa el nuevo texto sin reconstruir necesariamente el documento entero.

El modelo es distinto de «hacer una nueva petición MVC y cargar otra página», pero no hay que ocultar su dependencia de la conectividad. Interactive Server necesita mantener comunicación entre navegador y servidor para los eventos de C#. Si interrumpimos la conexión, las operaciones interactivas pueden dejar de responder aunque la última representación HTML permanezca visible.

Tampoco debemos confundir esto con **WebAssembly**. En un modelo Interactive WebAssembly, el código de componentes puede ejecutarse en el navegador, tras descargar el runtime y los recursos necesarios. En Interactive Server, el código del componente y su estado interactivo se procesan principalmente en el servidor. No creamos ni compilamos una aplicación WebAssembly independiente en este curso.

Un mapa conceptual de modos de renderizado

La terminología de .NET 8 distingue SSR estático, Interactive Server, Interactive WebAssembly e Interactive Auto. Podemos comprender las diferencias principales sin añadir proyectos nuevos. SSR estático responde con HTML desde el servidor, sin procesamiento de eventos de componente. Interactive Server mantiene la lógica de eventos del lado servidor y necesita conexión interactiva. Interactive WebAssembly sitúa la ejecución interactiva del componente en el navegador. Interactive Auto combina mecanismos según el estado y los recursos disponibles, añadiendo complejidad que no necesitamos en seis horas.

No confundas **el lugar donde se produce HTML inicialmente** con **el lugar donde se ejecutan después los eventos**. El prerenderizado permite representar HTML antes de disponer del modo interactivo plenamente conectado. Así, una pantalla puede mostrar contenido y continuar preparando

su interactividad. En P3.6 veremos un efecto relevante de este proceso sobre `OnInitializedAsync`.

Diagnosticar cuándo Blazor parece «un HTML muerto»

Si el botón de `/interactiva` no incrementa, empieza comprobando que has abierto esa ruta y no `/estatica`. Revisa la directiva `@rendermode InteractiveServer` y el contenido de `App.razor`. Confirma después que `Program.cs` registra `AddInteractiveServerComponents` y `AddInteractiveServerRenderMode`. En el navegador, observa si existen errores de carga del script o de conexión interactiva.

No conviertas cada problema en una corrección de JavaScript; el curso trata de entender la interactividad de C#. Y no elimines `@rendermode` sin saber qué comparación estás realizando: si lo haces, estarás cambiando el escenario de prueba, no necesariamente solucionando un fallo.

La respuesta didáctica esperada de P3.1 es muy concreta: **la misma forma de escribir `@onclick` no produce el mismo comportamiento bajo SSR estático y bajo Interactive Server**. Esta afirmación se apoya en dos rutas y una observación reproducible, no en un diagrama abstracto.

Profundización opcional: ventajas, costes y límites del modelo

Interactive Server puede resultar útil cuando queremos construir una interfaz con C# sin distribuir al navegador toda la lógica de componentes. Permite reutilizar servicios del servidor, integrar validación y evolucionar rápidamente una aplicación orientada a formularios o procesos internos. Sin embargo, el servidor mantiene circuitos y estado por conexiones activas; por tanto, debemos considerar recursos, reconexión, latencia y capacidad de servir varios usuarios.

Es tentador afirmar que Blazor «evita totalmente JavaScript». Sería inexacto: el cliente utiliza recursos JavaScript del framework, y ciertas integraciones con APIs del navegador pueden requerir interoperabilidad JavaScript. Nuestra decisión pedagógica consiste en **no desarrollar JS interop avanzado**, porque no aporta al objetivo de aprender primero componentes y eventos.

También es impreciso vender Interactive Server como alternativa siempre superior a MVC. Una pantalla que representa principalmente contenido y realiza operaciones ocasionales puede encajar de forma natural en MVC o Razor Pages. Una experiencia con selección, estado visual y numerosos eventos puede resultar más expresiva en componentes interactivos. Se trata de una decisión de arquitectura relacionada con la experiencia, el servidor, la conectividad y el modelo de mantenimiento, no de una competición entre tecnologías.

Referencias: Microsoft Learn, Blazor render modes en ASP.NET Core 8, y fundamentos de componentes Razor.

Laboratorio de razonamiento: qué parte de la interacción pertenece al servidor

Para analizar la interacción de forma rigurosa conviene separar **dónde se ejecuta la lógica**, **qué datos viajan** y **qué conserva el navegador**. En Interactive Server, el componente maneja el evento principalmente en el servidor y mantiene su estado en un circuito asociado a la conexión interactiva. El navegador no necesita conocer el algoritmo de incremento del contador como una función propia: recibe una representación actualizada. Esto es precisamente lo que el alumno observa al pulsar el botón de `/interactiva`.

La comparación con `/estatica` ayuda a evitar una conclusión frecuente pero incorrecta: «Blazor convierte todo C# en JavaScript». El componente estático puede ejecutar C# durante la representación inicial en el servidor, pero eso no significa que todos sus métodos queden disponibles como manejadores de evento en el navegador. Interactive Server, por su parte, no implica enviar el código del método `Incrementar` al navegador para ejecutarlo allí; el circuito recibe los eventos y el servidor procesa la lógica del componente.

Imaginemos que el método del contador no suma una unidad, sino que consulta el valor de una propiedad del componente y elige un mensaje. En `/interactiva`, tras el clic se volverían a evaluar las expresiones Razor que dependen del estado actualizado. En `/estatica`, el HTML ya enviado no puede actualizarse por ese manejador de evento sin un mecanismo adicional. La

causa no es que la expresión Razor de `@contador` tenga una sintaxis distinta, sino que el modo de renderizado no ha habilitado el ciclo interactivo de eventos.

La noción de circuito también afecta al aislamiento de usuarios. Dos navegadores con sesiones distintas no deberían compartir sin más la instancia de componente que almacena el contador. Un campo de instancia dentro de `@code` pertenece al estado de esa instancia; no es una variable `static` común para toda la aplicación. Conviene explicar esto antes de que aparezcan listas mutables en P3.4, porque de otro modo un alumno podría pensar que cada producto añadido automáticamente se hará visible para todos los demás.

El papel exacto de los tres lugares de configuración

Es útil dividir la configuración en tres responsabilidades. En `Program.cs`, `AddRazorComponents` y `AddInteractiveServerComponents` proporcionan la infraestructura. El mapeo con `MapRazorComponents<App>().AddInteractiveServerRenderMode()` informa al host de los endpoints y modos de componente admitidos. En `Interactiva.razor`, `@rendermode InteractiveServer` declara que **esa página concreta** requiere interactividad. Finalmente, `App.razor` incluye el script necesario para que el navegador participe en la experiencia de Blazor.

Podemos probar la necesidad conceptual de cada pieza con un diagnóstico, sin convertirlo en un ejercicio obligatorio de romper archivos. Si faltara el registro del modo interactivo, la página podría no disponer de la infraestructura para ejecutar eventos; si faltara la declaración de modo en la página, podríamos observar solo representación estática; si fallara la carga del script o del circuito, un botón preparado puede seguir apareciendo visualmente pero no producir cambios. Estos fallos no se resuelven inventando una acción MVC porque las páginas actuales no dependen de ese controlador.

Hay además una diferencia de alcance entre declarar un modo en la página y establecerlo globalmente. Si asignáramos Interactive Server a todo `Routes`, las páginas del árbol adoptarían la interactividad global, salvo excepciones diseñadas para ello. Hemos optado por un modo explícito por página para que el alumno pueda contrastar el comportamiento estático e interactivo dentro

de una sola aplicación. Es una decisión del laboratorio, no una prohibición general del modo global.

Navegación frente a interacción dentro de una página

Si seleccionamos el enlace `/interactiva` desde la raíz, el navegador entra en la página asociada a esa ruta. Después, los clics del contador modifican el texto **sin que tengamos que navegar a otra dirección**. En MVC era frecuente observar una nueva URL o una respuesta de formulario después de la acción. En Blazor conviene diferenciar el evento de navegación de un evento que solo modifica estado local.

Esto no quiere decir que la navegación de Blazor nunca utilice HTTP. La aplicación puede realizar peticiones completas, especialmente al abrir inicialmente una URL o cuando se requiere una carga de documento. Tampoco significa que todas las interacciones deban quedar dentro de un circuito: en una aplicación mayor podemos combinar navegación, formularios, servicios y endpoints. En M03 nos limitamos a distinguir estas operaciones para no confundir arquitectura con apariencia.

Un escenario de fallo de conexión razonado

Supongamos que una persona tiene abierta `/interactiva`, donde el contador muestra «Pulsaciones: 5». Se desconecta temporalmente de la red y pulsa otra vez. La interfaz puede conservar visualmente el último valor mientras no puede comunicar correctamente el evento. Esto no demuestra que el método C# haya fallado al sumar: es posible que ni siquiera haya llegado a ejecutarse. El primer diagnóstico debería comprobar la conexión interactiva, no alterar la expresión aritmética del contador.

Una vez restablecida la conectividad, el comportamiento depende del estado del circuito y de la capacidad de reconexión. El laboratorio no implementa estrategias avanzadas de recuperación, pero el alumno debe comprender que **la continuidad de la sesión interactiva tiene un coste operativo**. La diferencia con un HTML estático descargado es importante: aquel puede continuar siendo legible sin conexión, aunque no disponga de eventos C# interactivos.

Actividad de transferencia sin nuevo código

Redacta un cuadro con tres columnas: «situación», «qué puedo observar» y «qué comprobaría primero». Incluye estos casos: URL inexistente, página estática con botón que no incrementa, página interactiva con botón que sí incrementa, página interactiva que pierde conexión y nueva carga de página que reinicia el contador. Cada caso representa una responsabilidad distinta: rutas, modo de renderizado, ciclo de eventos, conectividad y vida de la instancia.

El resultado correcto del ejercicio no es memorizar el nombre de SignalR ni diagnosticar automáticamente todo como «problema de Razor». Debes poder justificar por qué una modificación en `Program.cs` sería relevante en algunos casos y completamente innecesaria en otros. Esta capacidad de diagnóstico es el puente hacia el resto de M03, donde los mismos conceptos se aplicarán al estado de un catálogo de productos.

3.2. Estructura de Blazor Web App, componentes, páginas y navegación

El proyecto B como unidad de aprendizaje

La estructura de un proyecto nuevo suele intimidar porque aparecen archivos que no existían en la solución MVC. La forma de entenderla no es memorizar una lista, sino seguir la petición hasta el componente que se representa. La aplicación B contiene `Program.cs`, `Components/App.razor`, `Components/Routes.razor`, `Components/Layout/MainLayout.razor`, `Components/Pages`, `wwwroot` y, a partir de 3.2, un pequeño modelo de productos. Todas esas piezas existen para separar el arranque del host, la composición de la interfaz y los datos que representa.

En M01 aprendimos que `Program.cs` configura servicios y endpoints. Esa idea no cambia en Blazor Web App: seguimos dentro de ASP.NET Core. Lo que cambia es que el endpoint principal representa componentes Razor, no una vista elegida por una acción MVC. Una página como `/productos` la declara un archivo `.razor` con una directiva `@page`, y el router de Blazor participa en seleccionar ese componente.

App.razor y el documento inicial

El archivo `Components/App.razor` incorpora la estructura del documento:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8" />
    <base href="/" />
    <link rel="stylesheet" href="app.css" />
    <HeadOutlet />
</head>
<body>
    <Routes />
    <script src="_framework/blazor.web.js"></script>
</body>
</html>
```

`HeadOutlet` participa en el contenido que se inserta en la cabecera, por ejemplo el título de página. `Routes` contiene el mecanismo de representación de

componentes vinculados a direcciones. El elemento `base` fija la base de resolución de rutas relativas y recursos. La hoja de estilos se distribuye desde `wwwroot/app.css`; es un recurso estático y no es necesario crear un controlador para servirla.

Aunque `App.razor` tiene formato de HTML, **no es una vista MVC**. Es el componente raíz de la aplicación y su composición sirve de contenedor para el árbol de componentes. En nuestro laboratorio elegimos mantener `Routes` sin modo global para que cada página pueda optar por Interactive Server donde sea necesario.

Routes.razor y selección de página

El archivo `Components/Routes.razor` incluye:

```
<Router AppAssembly="typeof(Program).Assembly">
    <Found Context="routeData">
        <RouteView RouteData="routeData"
                   DefaultLayout="typeof(MainLayout)" />
        <FocusOnNavigate RouteData="routeData" Selector="h1" />
    </Found>
    <NotFound>
        <LayoutView Layout="typeof(MainLayout)">
            <h1>404 | No existe la página solicitada</h1>
        </LayoutView>
    </NotFound>
</Router>
```

El router identifica qué componente responde a la ruta; `RouteView` lo representa y aplica el layout predeterminado. La rama `NotFound` ofrece una respuesta visible cuando no existe una página de componente correspondiente. `FocusOnNavigate` ayuda a gestionar el foco cuando navegamos, lo que facilita la accesibilidad básica sin añadir lógica de navegación manual.

En MVC escribimos rutas como `/Productos/Detalle/2` y un controlador recibía el parámetro. En Blazor, los componentes de página se identifican con `@page`. La similitud de nombres «Page» no implica que estemos usando Razor Pages: aquí se trata de un componente `.razor` administrado por el router de Blazor, no un archivo `.cshtml` respaldado por `PageModel`.

Una nueva página Productos

El archivo `Components/Pages/Productos.razor`, entregado en el estado 3.2, comienza con:

```
@page "/productos"
@rendermode InteractiveServer
<PageTitle>Productos</PageTitle>
<h1>P3.2 | Página Productos</h1>
```

La directiva `@page "/productos"` relaciona el componente con esa dirección. `@rendermode InteractiveServer` prepara la página para evolucionar hacia eventos posteriores. En este punto el contenido es principalmente una lista, de modo que el modo interactivo todavía no produce un comportamiento visual diferente; lo dejaremos configurado para construir sobre la misma idea de componente en 3.3–3.5.

Esta es una decisión para reducir transiciones conceptuales: el alumno aprende la ruta y el archivo antes de añadir parámetros, eventos y callbacks. La configuración es real y no un comentario que «se activará mágicamente» cuando añadamos un botón.

Layout y navegación

`Components/Layout/MainLayout.razor` es un componente de layout. Incluye la directiva `@inherits LayoutComponentBase` y un lugar donde se inserta `@Body`. En ese mismo archivo se representan enlaces mediante `NavLink`, que conoce la dirección de navegación y puede aplicar un estado de enlace activo. En P3.2 se activa el bloque:

```
<NavLink href="/productos">Productos</NavLink>
```

Este fragmento no añade una ruta por sí mismo. La ruta ya la declaró la página mediante `@page`; el menú únicamente ofrece una forma visible de llegar a ella. Por tanto, si escribir directamente `/productos` funciona pero el menú no muestra el enlace, debemos investigar el layout. Si el enlace aparece pero la dirección devuelve una página inexistente, revisaremos `@page`, la ubicación del componente y el router.

El paralelismo con MVC es útil: el layout ofrece una estructura común y evita repetir navegación en todas las pantallas. La diferencia es que ahora el layout también es un componente Razor, no una vista `_Layout.cshtml`. No utilizamos `RenderBody` de MVC: usamos `@Body` del componente de layout.

Imports y visibilidad de componentes

El archivo `Components/_Imports.razor` reúne directivas `@using` relevantes. El proyecto utiliza, entre otros, `Microsoft.AspNetCore.Components.Web` y `Microsoft.AspNetCore.Components.Routing`, así como los espacios de nombres de la propia aplicación. Esto reduce la repetición de `@using` en cada componente. Sin embargo, no crea objetos ni registros de dependencia: importar un namespace solo facilita localizar tipos en el código.

Una incidencia pedagógica habitual surge cuando alguien pega un componente en otra carpeta y el compilador no encuentra su etiqueta o tipo. Antes de renombrar arbitrariamente el componente, conviene comprobar el espacio de nombres, los imports y si el archivo forma parte del proyecto correcto. Una etiqueta Razor de componente tiene un significado diferente del nombre de un elemento HTML ordinario.

Modelos en memoria y sintaxis Razor

En `Models/Producto.cs` utilizamos un record de C#:

```
namespace CursoWeb.Blazor.Models;

public sealed record Producto(int Id, string Nombre, decimal Precio);
```

No enseñamos aquí qué es un record desde cero, porque el alumnado ya domina C#. Lo empleamos para conservar un contrato sencillo. En la página, una colección de productos permite iterar utilizando Razor:

```
<ul id="lista-productos">
    @foreach (var producto in productos)
    {
        <li>@producto.Nombre —
            @producto.Precio.ToString("0.00") €</li>
```

```
    }
</ul>
```

`@foreach` se ejecuta cuando el componente se representa. El navegador recibe elementos de lista HTML, no una instrucción C# que deba entender. El alumno puede inspeccionar tres `li` resultantes y relacionarlos con tres instancias de `Producto`. Después, en 3.3, sustituiremos conceptualmente el listado simple por tarjetas reutilizables.

No hay llamada a API REST ni un DbContext: los tres objetos se crean en memoria como datos iniciales. Eso limita la duración del estado y evita añadir una dependencia externa a la comprensión de routing y marcado.

Flujo completo de una navegación

Supongamos que desde el inicio seleccionamos «Productos». El navegador sigue el enlace hacia `/productos`. El host ASP.NET Core reconoce los endpoints de componentes, el router identifica `Productos.razor` por `@page`, la página se representa dentro de `MainLayout` y genera una lista a partir del modelo. No hemos tenido que crear `ProductosController.Index`, aunque el nombre visible de la pantalla sea el mismo que en el módulo anterior.

Para entenderlo correctamente conviene distinguir un **componente routable** de un **componente reutilizable**. El primero declara `@page` y puede corresponder a una URL. El segundo, que introduciremos en 3.3, no tiene por qué declarar una ruta: puede representarse como etiqueta dentro de muchos componentes padre. Esta distinción es central para diseñar una aplicación Blazor mantenible.

Errores y diagnóstico

Si el proyecto compila pero `/productos` no aparece en el menú, comprueba el enlace preparado en `MainLayout.razor` y si has retirado los delimitadores correctos de comentario Razor. Si el enlace se ve pero devuelve una página 404, comprueba que `Productos.razor` contiene `@page "/productos"` y no has cambiado su dirección sin actualizar la navegación.

Si aparece una pantalla en blanco después de pegar un componente, revisa primero la salida de compilación y el registro de errores del navegador. No interpretes todo problema de Razor como un fallo de HTML: una excepción de C# durante el renderizado, un tipo no resuelto o un modo interactivo mal configurado también pueden impedir la representación.

Si un cambio del menú parece no aplicarse, comprueba que ejecutas el proyecto de la carpeta de **P3.2** y no la versión de P3.1 que también está disponible en el repositorio. Es una distinción importante porque hemos elegido estados docentes independientes para reproducir cada ejercicio sin destruir el trabajo anterior.

Profundización opcional: organización y estabilidad

En aplicaciones mayores, las páginas no necesitan contener toda la lógica, los modelos y los servicios en un único archivo. Podemos extraer componentes, inyectar servicios, crear modelos de presentación y mantener los archivos pequeños. Sin embargo, anticipar en P3.2 una arquitectura de docenas de servicios sería contrario al propósito de comprender la estructura. La evolución 3.2→3.3→3.4 demuestra **por qué** se extraen responsabilidades antes de introducirlas por costumbre.

Mantener una URL estable también forma parte del diseño. Cambiar `@page "/productos"` puede romper marcadores, enlaces externos o pruebas, aunque la aplicación siga compilando. La navegación visible en el layout y las rutas de los componentes deben mantenerse coherentes. Esta observación es transferible a MVC, Razor Pages y Blazor: los mecanismos cambian, pero una URL continúa siendo un contrato público.

Referencia: Microsoft Learn, Routing y navegación de Blazor.

Profundización guiada: cómo se resuelve una ruta de componente

Una dirección como `/productos` es una promesa: cuando alguien la solicita, nuestra aplicación debe ser capaz de identificar el contenido correspondiente. En el proyecto B, la directiva `@page "/productos"` anuncia que `Productos.razor` es un

componente routable. El router lo localiza en el ensamblado de la aplicación, y `RouteView` se ocupa de representarlo dentro del layout. Esta cadena se puede recorrer leyendo tres archivos, sin requerir un controlador MVC adicional.

Comparemos con la solución de M01. Allí la ruta `/Productos/Detalle/2` podía seleccionar `ProductosController.Detalle` y devolver una vista tipada. El controller estaba concebido como elemento de coordinación HTTP. En Blazor, el componente de página puede incorporar además su propia representación y el estado que utiliza; la ruta está asociada al componente, no a una acción nombrada como `Index`. Es una diferencia de organización y de comportamiento disponible, no una afirmación de que el framework haya dejado de usar ASP.NET Core.

App, Routes y Layout como capas de composición

`App.razor` define la estructura del documento y coloca el componente `Routes` donde se representa la parte variable de la aplicación. `Routes.razor` establece cómo seleccionar una página según la URL y aplicar un layout. `MainLayout.razor` define una estructura visual común con navegación y `@Body`. La página Productos suministra el contenido específico de su ruta, y más adelante la tarjeta suministrará un fragmento reutilizable de ese contenido.

Podemos comprenderlo como una composición de responsabilidades. El componente raíz no necesita conocer los tres productos; el router no necesita saber cuánto cuesta Lápiz; el layout no necesita gestionar el campo de selección; y la tarjeta no necesita conocer todas las rutas de la aplicación. Cada pieza tiene una finalidad y un contrato que podemos identificar.

Un error de principiante es trasladar todo el código del catálogo a `App.razor` porque allí aparece la etiqueta HTML `body`. Esa ubicación no es apropiada para un comportamiento que pertenece a la página Productos. Otro error es declarar navegación en un componente reutilizable que debería limitarse a mostrar un producto. Los ejemplos del curso evitan estos diseños: la página gestiona la colección, el layout ofrece navegación y la tarjeta recibe parámetros.

Comprender el carácter de NavLink

`NavLink` es un componente que produce navegación y reconoce cuándo su dirección corresponde a la ruta actual. Esto permite marcar visualmente la sección activa del menú. Pero su existencia no implica que el router haya registrado automáticamente el destino. En P3.2 comprobamos primero la ruta directa y solo después activamos el enlace. La secuencia importa porque permite aislar dos condiciones: la página puede existir sin enlace de navegación; un enlace puede existir pero llevar a una ruta no registrada.

Si un alumno modifica `href="/productos"` a `href="/producto"` sin cambiar la directiva `@page`, es probable que termine en una página no encontrada. Si cambia solo el texto «Productos» por «Catálogo», la ruta no tiene por qué cambiar. Estas pruebas conceptuales ilustran que el texto visible, la dirección pública y el archivo de implementación son tres cosas distintas que se coordinan, pero no se sustituyen.

La extensión .razor no basta para crear una página

Un componente como `TarjetaProducto.razor` puede existir sin @page; se usa desde otro componente por su nombre de etiqueta. Una página como `Productos.razor` sí declara una ruta. Este contraste prepara la separación de P3.3. En una aplicación mayor puede haber cientos de componentes reutilizables y solo un subconjunto de ellos ser routables. Pensar que todos los archivos `.razor` son direcciones abriría un modelo erróneo de navegación.

Tampoco debemos asimilar una ruta de componente a una Razor Page del módulo anterior. Ambas pueden utilizar la directiva `@page`, pero los archivos tienen extensiones y modelos de ejecución distintos. La Razor Page `Comparacion.cshtml` se organiza con `PageModel` y handlers HTTP; el componente `Productos.razor` participa en el árbol de componentes y puede mantener estado interactivo.

Sintaxis Razor y responsabilidades de datos

Cuando escribimos `@foreach (var producto in productos)` dentro del componente, Razor genera una representación para cada objeto. La colección inicial pertenece al campo declarado en `@code`, no a un modelo que una acción MVC

haya pasado a una vista. No necesitamos crear `ViewData` ni `ViewBag` para que esa variable exista. No obstante, sigue siendo C#; si la colección contiene un elemento con precio 3.75, `ToString("0.00")` controlará la cadena de precio representada.

La distinción entre nombre del objeto y texto visible también importa. Cambiar la etiqueta del menú no renombra la propiedad `Nombre`. Cambiar el nombre de una propiedad del record sí exige adaptar las expresiones que la utilizan. La herramienta de compilación puede ayudarnos a localizar referencias rotas, pero el navegador continúa siendo necesario para observar cómo se representa el resultado.

Caso de transferencia: una ruta correcta, una pantalla sin productos

Un compañero comunica que puede abrir `/productos` y ve el título, pero no aparecen los tres elementos. Como la ruta existe, no tiene sentido empezar creando otro `NavLink`. Debemos revisar la colección dentro de `Productos.razor` y la condición o iteración que produce los elementos. Si estuviera vacía, no hay nada que representar; si la vista estuviera mostrando una variable equivocada, la salida tampoco reflejaría los datos. Esta es una incidencia de estado y marcado, no de routing.

En cambio, si la dirección devuelve 404, revisar los datos de Cuaderno es prematuro. Primero comprobamos la directiva `@page`, la ubicación del archivo y el router. Distinguir estas causas es una habilidad que se reutilizará cuando la página pase a depender de un servicio asíncrono en 3.6.

Actividad de interpretación de proyecto

Elabora un recorrido escrito que empiece con `http://localhost:5092/productos` y termine en el texto «Lápiz — 1.20 €». Nombra `Program.cs`, `App.razor`, `Routes.razor`, `MainLayout.razor`, `Productos.razor` y `Producto.cs`, asignando una sola responsabilidad principal a cada uno. La evidencia está en los archivos existentes; no hace falta agregar un servicio, una API ni otra vista.

Después describe qué archivo modificarías si: deseas cambiar el título de la ventana, quieres añadir un enlace de menú, necesitas crear una ruta nueva, deseas cambiar el precio de Lápiz en el estado fijo o quieres modificar el estilo

de todos los elementos. La respuesta correcta será diferente en cada caso, porque se trata de responsabilidades diferentes. Esta reflexión prepara la extracción de componentes de P3.3.

3.3. Parámetros, reutilización y composición de componentes

Por qué extraer una tarjeta del listado

Al terminar 3.2, `Productos.razor` puede mostrar tres registros recorriendo una colección. La solución funciona, pero la pantalla podría crecer: cada producto necesita título, precio, referencia, estilo, acciones y una posible representación en diferentes páginas. Si repetimos el mismo bloque en cada ubicación, cualquier cambio visual obligará a editar varias copias. Un **componente reutilizable** encapsula una pieza de interfaz con una responsabilidad comprensible y permite utilizarla varias veces con datos distintos.

En P3.3 extraemos `Components/TarjetaProducto.razor`. No lo transformamos en una página navegable: no tiene `@page`. Su función es recibir un producto y representarlo. La página Productos seguirá siendo responsable de decidir **qué productos** aparecen y en qué orden. La tarjeta se ocupa de **cómo se muestra cada producto individual**.

Esta distinción entre composición y navegación es fundamental. En MVC aprendimos a usar vistas parciales para reutilizar marcado. Un componente Razor cumple también una función de reutilización, pero forma parte del modelo de componentes de Blazor y puede desarrollar parámetros, estado y eventos propios. En 3.5 veremos una capacidad adicional: enviar un evento tipado de vuelta al padre.

El contrato del componente

El archivo de la tarjeta comienza así:

```
<article class="tarjeta" data-producto-id="@Producto.Id">
    <h2>@Producto.Nombre</h2>
    <p>Precio: @Producto.Precio.ToString("0.00") €</p>
    <small>Referencia @Producto.Id</small>
</article>

@code {
```

```
    [Parameter, EditorRequired]
    public Producto Producto { get; set; } = default!;
}
```

`[Parameter]` indica que la propiedad puede recibir un valor desde el componente que utiliza la tarjeta. `EditorRequired` ayuda a las herramientas a señalar que el autor espera que ese parámetro se suministre. No es un validador HTTP ni sustituye el razonamiento sobre nulos en tiempo de ejecución. El inicializador `default!` comunica al análisis de nulabilidad que el valor se recibirá desde el padre, sin crear una instancia ficticia de producto.

El contrato público es pequeño: una propiedad `Producto` de un tipo conocido. No pasamos tres cadenas sin relación ni hacemos que la tarjeta consulte por su cuenta un servicio que todavía no existe. Conservar el objeto completo facilita que el componente muestre campos coherentes pertenecientes al mismo registro.

Cómo pasa el padre un objeto al hijo

La variante activable de la página contiene:

```
<div class="tarjetas" id="tarjetas-productos">
    @foreach (var producto in productos)
    {
        <TarjetaProducto Producto="producto" />
    }
</div>
```

La etiqueta `TarjetaProducto` no es un elemento HTML estándar. Hace referencia al componente Razor. El atributo `Producto="producto"` asigna el objeto actual de la iteración a la propiedad marcada con `[Parameter]`. Se crean tantas instancias de la tarjeta como elementos contiene la colección.

El nombre del atributo corresponde a la propiedad pública del componente; el valor procede de la variable `producto` del ámbito del `foreach`. Si se invierte o se pasa una expresión incompatible con el tipo esperado, el proyecto puede dejar de compilar. La ventaja es que el contrato tiene semántica de C# y se puede inspeccionar, no solo que el HTML quede más bonito.

Cambiar el origen de datos sin cambiar la tarjeta

La tarjeta no necesita saber si el objeto procede de una lista escrita en el propio componente, de un servicio o de una API futura. Mientras reciba una instancia válida de `Producto` podrá representarla. En 3.6 moveremos la obtención de productos a un servicio en memoria; en M04 trabajaremos con HTTP y DTO. La tarjeta podrá seguir enfocada en mostrar el producto, aunque quizá adaptemos los tipos de datos al nuevo contrato.

Este punto anticipa una virtud de la **separación entre composición y adquisición de datos**: el componente de representación tiene menos dependencias, por lo que es más sencillo comprenderlo y probar su resultado. La ventaja no exige introducir arquitecturas empresariales o jerarquías complejas de interfaces durante P3.3.

La dirección de los datos

En esta práctica los datos fluyen desde el padre hacia el hijo. El padre posee la lista; el hijo recibe un elemento. Si modificamos el objeto en el padre y se vuelve a representar la tarjeta, el componente debe reflejar los valores recibidos. Eso no significa que el hijo deba encargarse de mutar la colección del padre. En 3.5 estudiaremos un contrato explícito para comunicar una intención desde una tarjeta hacia su contenedor.

Un error habitual consiste en asignar un nuevo valor a una propiedad `[Parameter]` dentro del hijo y esperar que eso cambie por sí mismo la fuente de datos del padre. El componente padre continúa siendo dueño de la lista; un parámetro representa entrada del componente, no un sistema automático de sincronización bidireccional.

Qué hacemos con el estado inicial del proyecto

El estado de P3.3 contiene tanto la lista sencilla activa como la variante de tarjetas comentada. Esto permite comprobar primero un proyecto funcional y, después, realizar una transformación guiada. La práctica ordena comentar el bloque `<ul> ... </ul>` y activar el bloque `<div class="tarjetas">`. Si se activaran los

dos simultáneamente, aparecerían dos representaciones del mismo catálogo; eso sería un resultado explicable, pero no el objetivo de la práctica.

Aquí hay una decisión didáctica clara: no se entrega una aplicación que solo funciona cuando todo el material se descomenta. El estado inicial ya puede compilarse y mostrar los productos, y la transición puede verificarse como un cambio concreto de presentación.

Composición frente a herencia

La tarjeta puede ser un componente pequeño y autocontenido porque **compone** la interfaz: está dentro de la página Productos y no hereda su contenido. No necesitamos construir una clase base para todas las páginas ni una jerarquía de componentes «ProductoBase», «TarjetaEspecializada» y «TarjetaCompleta». La composición favorece decisiones locales: el padre controla la colección y el hijo se concentra en una representación de elemento.

Podemos comparar esto con las vistas parciales de MVC. Una parcial también ayuda a evitar marcado duplicado y puede recibir un modelo; en Blazor, además, el componente es una unidad que participa en parámetros, ciclo de representación y eventos. La similitud sirve como puente pedagógico, pero no convierte sus modelos de ejecución en idénticos.

Observaciones al depurar parámetros

Si todas las tarjetas muestran «Cuaderno», abre primero la página padre y comprueba que pasas `producto` de cada iteración, no `productos[0]`. Después revisa la propiedad `[Parameter]` en la tarjeta. Si una tarjeta muestra la referencia de un producto y el precio de otro, es posible que hayas compuesto valores procedentes de objetos diferentes; pasar un `Producto` completo reduce este riesgo, pero debes conservar la expresión correcta en el marcado.

Si aparece un mensaje de componente desconocido, comprueba que `TarjetaProducto.razor` existe en `Components` y que el espacio de nombres está disponible mediante `_Imports.razor`. Si el compilador se queja de un tipo de

parámetro, revisa la propiedad pública y lo que se transmite desde el padre; no cambies arbitrariamente el modelo de dominio para silenciar el error.

Si las tarjetas se representan sin estilos, la lógica del parámetro puede estar funcionando: inspecciona `class="tarjeta"` y comprueba que se ha cargado `wwwroot/app.css`. No es necesario reconstruir la lógica C# para solucionar un CSS ausente.

Profundización opcional: identidad y cambios de listas

Cuando una lista cambia de orden o se eliminan elementos, el sistema de representación de Blazor necesita asociar estructuras anteriores y nuevas de forma coherente. En componentes de listas más dinámicas puede resultar útil la directiva `@key` con una identidad estable. No es obligatorio introducirla para nuestro catálogo fijo de tres productos de P3.3, pero conviene conocer el problema que resuelve: conservar la asociación entre un elemento de datos y su componente cuando la colección se reorganiza.

Esto se diferencia del atributo HTML `id`, que identifica elementos del documento, y del campo `Producto.Id`, que identifica el producto en el dominio. `@key` es una indicación al motor de representación de componentes. No debe utilizarse como sustituto de un identificador de producto correctamente diseñado.

También podemos preguntarnos cuándo resulta contraproducente extraer un componente. Si una pieza consiste en una etiqueta que solo aparece una vez y no tiene una responsabilidad independiente, quizá la extracción complique más de lo que mejora. La reutilización se justifica cuando ofrece un contrato claro, concentra variaciones y facilita comprender la interfaz. Nuestra tarjeta cumple esas condiciones porque aparece varias veces y seguirá participando en eventos en 3.5.

Referencia: Microsoft Learn, Parámetros de componentes.

Profundización aplicada: diseñar componentes a partir de responsabilidades

La extracción de `TarjetaProducto` puede parecer un gesto meramente estético si solo observamos que las tarjetas tienen bordes y títulos. Pero su valor principal está en definir un **contrato de representación**: el padre suministra un producto y el hijo representa sus propiedades. Esta separación permite cambiar el modo de presentación de todos los productos desde un solo archivo, y también deja claro quién controla el catálogo.

No tenemos aquí una capa de dominio compleja. El record `Producto` contiene tres valores fáciles de reconocer. El componente no necesita saber si el precio procede de una base SQL, de un fichero o de un servicio en memoria, siempre que reciba el objeto esperado. Esta independencia será útil cuando en P3.6 introduzcamos un servicio y, posteriormente, en M04 una API. La interfaz de la tarjeta puede seguir estable mientras cambia el mecanismo de obtención de datos.

La diferencia entre referencia de objeto y copia de texto

Podríamos diseñar TarjetaProducto con tres parámetros separados: `Id`, `Nombre` y `Precio`. Sería una solución técnicamente posible, pero exigiría que el padre transmitiera y mantuviera alineados los valores por separado. El proyecto usa un único `[Parameter] public Producto Producto`, que hace explícita su pertenencia al mismo objeto. Esta elección reduce las combinaciones incorrectas que pueden crearse por descuido.

Esto no impide que existan componentes con parámetros simples. Un título, un mensaje o un indicador booleano pueden ser perfectamente adecuados como parámetros individuales. La decisión relevante es expresar un contrato coherente para la responsabilidad del componente, no convertir todas las propiedades en un objeto gigantesco.

Ciclo de representación y parámetros

Cada vez que el padre necesita actualizar su representación, puede proporcionar nuevos valores a sus componentes hijos. Los hijos no deben asumir que el parámetro recibido se mantendrá inmutable durante toda la

vida de la página si el padre cambia de datos. De manera parecida, modificar la propiedad pública marcada `[Parameter]` en el hijo no debe utilizarse como una estrategia oculta para reescribir el estado del padre. El canal explícito de eventos se estudiará en P3.5.

Un buen componente de presentación recibe datos y calcula HTML a partir de ellos. Cuando necesita comunicar una acción, declara un callback o contrato de evento. Estas dos operaciones tienen direcciones distintas y representan responsabilidades distintas. El alumnado debería poder explicar esa distinción antes de introducir botones de tarjeta en el siguiente punto.

Cómo localizar un error de composición

Considera este fragmento hipotético:

```
@foreach (var producto in productos)
{
    <TarjetaProducto Producto="productos[0]" />
}
```

La iteración recorre tres elementos, pero el valor entregado al hijo es siempre el primero. El resultado serán tres tarjetas iguales. No hay error de compilación, porque la expresión tiene el tipo correcto. La incorrección es **semántica**: el dato de cada iteración no se transmite de forma coherente. Por eso la prueba de P3.3 compara los identificadores de las tarjetas, no solo cuenta que existan tres.

Ahora compara la variante correcta:

```
@foreach (var producto in productos)
{
    <TarjetaProducto Producto="producto" />
}
```

El componente hijo no necesita adivinar qué Id corresponde a cada posición: lo recibe. El HTML resultante puede identificarse en el navegador mediante `data-producto-id`, pero el atributo de diagnóstico no es el mecanismo de paso de parámetro. Es una consecuencia visible del objeto que recibe el hijo.

Reutilización de marcado sin duplicar responsabilidades

Si queremos cambiar el texto «Precio:» por «Importe:», conviene modificar TarjetaProducto. Si queremos ordenar los productos, conviene modificar la colección o la lógica del padre. Si queremos añadir una cuarta fila de datos al estado fijo, modificamos la lista del padre. Si queremos marcar visualmente ciertos precios, el hijo puede incluir una condición Razor basada en `Producto.Precio`.

Estos ejemplos justifican por qué no toda modificación debe realizarse en el mismo archivo. Una arquitectura didáctica de dos componentes ya puede enseñar separación de responsabilidades. No necesitamos anticipar sistemas de plantillas, bibliotecas de componentes empresariales o una estructura de docenas de proyectos para obtener una mejora real de mantenimiento.

Ejercicio comparativo con vistas parciales MVC

En M01 utilizamos vistas parciales para representar productos. Un paralelismo útil es que ambas técnicas reducen repetición de marcado y reciben datos. Pero en Blazor, los componentes participan además en un ciclo de representación, admiten parámetros propios y pueden incorporar manejadores de eventos. En MVC, una vista parcial no se convierte simplemente en un componente interactivo por cambiar su nombre; pertenece al proceso de representación de vistas que inicia una acción.

Al describir ambas técnicas, evita frases ambiguas como «Blazor es lo mismo que Razor, pero moderno». La sintaxis Razor es compartida, pero la unidad de composición y el modelo de ejecución son distintos. Esta diferencia se volverá evidente cuando TarjetaProducto solicite al padre una selección mediante EventCallback.

Actividad de transferencia sin introducir nueva arquitectura

Supongamos que quieres mostrar los tres productos en dos ubicaciones: una página de catálogo y un pequeño resumen informativo. ¿Qué reutilizarías? La tarjeta puede formar parte de ambas composiciones si recibe el mismo contrato de producto. ¿Qué no debería hacer la tarjeta? No debería decidir

qué productos existen, ni consultar por sí misma la base de datos del M02, ni asumir que siempre forma parte de una página denominada `/productos`.

En tu respuesta, nombra los datos de entrada, la salida visual y cualquier dependencia que sería innecesaria. Si puedes defender por qué el hijo conoce un producto pero no la lista completa, has comprendido el fundamento de la composición que usaremos en los eventos de P3.5.

3.4. Eventos, enlace de datos y estado de componente

El paso decisivo: de mostrar productos a interactuar con ellos

Los ejercicios anteriores han producido HTML y componentes reutilizables, pero su contenido todavía era mayoritariamente una representación de datos. Ahora queremos que el alumno **seleccione un producto**, **introduzca un nombre** y **añada un elemento a la lista**, todo sin construir un nuevo formulario MVC ni introducir una API remota. Este paso concentra buena parte del valor formativo de Blazor: comprender cómo un evento altera un campo C# y provoca una nueva representación visible de la interfaz.

El estado inicial de P3.4 se ejecuta con `@rendermode InteractiveServer`. La página ya muestra una lista de productos y un texto «Seleccionado: ninguno», pero los botones de selección y creación están comentados para permitir una evolución controlada. Esta estructura evita un error común de los ejemplos docentes: presentar únicamente el código final y pedir al estudiante que deduzca las transiciones intermedias.

Estado local: qué es y qué no es

En `Productos.razor` encontramos:

```
private readonly List<Producto> productos =
[
    new(1, "Cuaderno", 4.50m),
    new(2, "Lápiz", 1.20m),
    new(3, "Carpeta", 3.75m)
];

private string seleccion = "ninguno";
```

Los campos pertenecen a una instancia del componente. `productos` almacena referencias a los productos disponibles durante la vida de esa instancia; `seleccion` conserva una representación de la elección actual. No estamos utilizando una columna SQL, una cookie, un almacenamiento local del navegador ni un singleton global. La propiedad `readonly` impide reasignar el

campo de lista, pero no impide modificar su contenido con `Add`; ese matiz procede de C# y aquí lo aplicamos a la interfaz.

El estado local resulta adecuado para una interacción limitada al componente. Si necesitáramos que muchas páginas compartieran la misma selección, tendríamos que diseñar otra ubicación para el estado. No adelantamos ese problema porque P3.4 busca que se reconozca primero la relación entre **campo**, **evento** y **representación**.

Un botón que invoca un método C#

La activación del paso 3.4.2 añade al marcado de cada elemento:

```
<button @onclick="() => Seleccionar(producto.Id)">
    Seleccionar
</button>
```

Y añade al bloque `@code` el método:

```
private void Seleccionar(int id)
{
    seleccion = productos.First(p => p.Id == id).Nombre;
}
```

La expresión lambda permite transportar el Id del producto que corresponde al botón seleccionado. Al pulsarlo, Blazor ejecuta `Seleccionar`, encuentra el objeto y cambia `seleccion`. El texto representado en `<p id="seleccion">` se actualiza porque el framework vuelve a representar el componente después de procesar el evento. No necesitamos llamar manualmente a `StateHasChanged()` para esta operación normal de evento de componente.

La distinción respecto de MVC es inmediata. No escribimos una acción `[HttpPost]` ni devolvemos `View(producto)`: el evento modifica el estado en la misma sesión interactiva. Tampoco podemos suponer que será persistente; una nueva instancia de la página puede comenzar con «ninguno» otra vez.

Por qué usamos una lambda y no ejecutamos el método al representar

Es importante distinguir **referenciar un manejador** de **invocar un método durante el renderizado**. `@onclick="() => Seleccionar(producto.Id)"` es una función que Blazor puede llamar cuando ocurre el clic. El alumno debe comprender que `producto.Id` procede del elemento actual de la lista. Una firma incompatible o una expresión incorrecta podría impedir la compilación o producir una selección diferente de la esperada.

Este razonamiento también ayuda a depurar cuando todas las filas responden como si fueran el mismo producto. En el recorrido actual, cada botón se genera con el Id del producto de su iteración. No hay un único enlace compartido por accidente ni una dirección HTTP que tenga que reconstruirse después.

Enlace de datos mediante @bind

El paso 3.4.3 introduce un campo para escribir el nombre de un nuevo producto:

```
<label>Nombre nuevo:
    <input id="nombre-nuevo" @bind="nuevoNombre" />
</label>
```

Con su campo de respaldo:

```
private string nuevoNombre = string.Empty;
```

`@bind` relaciona el valor del control con un miembro del componente. Cuando el usuario cambia el valor según el evento de enlace previsto, Blazor asigna el texto a `nuevoNombre`. Esto es diferente de un `asp-for` de MVC: aquel Tag Helper construía atributos de HTML para un formulario enviado mediante HTTP; aquí estamos trabajando con el estado de un componente interactivo.

En un elemento HTML `input`, el enlace predeterminado suele actualizar el valor del componente con el evento de cambio, no necesariamente en cada pulsación de tecla. Si un alumno desea observar cambios inmediatos, puede

estudiar posteriormente `@bind:event="oninput"`, pero el itinerario principal no necesita introducir esa variante antes de entender el comportamiento básico.

Añadir un objeto a una lista en memoria

En 3.4.4 se activa el botón:

```
<button id="agregar" @onclick="Agregar">
    Añadir producto
</button>
```

Y su código, junto con un contador interno de Id:

```
private int siguienteId = 4;

private void Agregar()
{
    if (string.IsNullOrWhiteSpace(nuevoNombre))
        return;

    productos.Add(new Producto(siguienteId++, nuevoNombre.Trim(), 5m));
    nuevoNombre = string.Empty;
}
```

El método rechaza de forma sencilla un nombre vacío o compuesto de espacios. Después crea un producto con precio fijo de laboratorio, lo agrega a la lista y limpia el campo para el siguiente alta. Hemos elegido un precio constante porque **no queremos construir un segundo CRUD ni repetir en M03 todas las validaciones de EF Core**: el objetivo de este paso es que el alumno observe inmediatamente una nueva tarjeta en la interfaz y comprenda cómo se ha producido el cambio.

El identificador incremental solo se declara cuando se activa este paso, de modo que el estado inicial compila sin tener variables docentes que todavía no se utilizan. Los ejemplos y retos conservarán siempre esa disciplina: una transición programable debe activar todos los bloques de los que depende.

Estado positivo, estado negativo y límites

Con el nombre «Regla» escrito en el campo, el botón Añadir incorpora una nueva tarjeta en la lista. Si el campo está vacío, no aparece una cuarta tarjeta. Esta diferencia sirve de comprobación funcional sencilla. No confundas el hecho de que la pantalla represente un cuarto elemento con que exista una nueva fila SQL: no hay DbContext en el proyecto ni se envía una petición de creación a una API.

Si refrescamos la página de manera que se crea un componente nuevo, la lista puede volver al conjunto inicial de tres productos. Ese comportamiento no constituye pérdida inesperada de datos persistentes; es la consecuencia del **estado local de componente** que hemos elegido deliberadamente. El aprendizaje es reconocer qué alcance tiene cada dato, no obtener permanencia de un mecanismo que nunca la prometió.

Retos y variantes de interacción

El reto 3.4 prepara un botón que ejecuta `Limpiar`:

```
private void Limpiar() => seleccion = "ninguno";
```

Se activa junto con su botón Razor de «Limpiar selección». Este reto tiene un resultado verificable: después de seleccionar Lápiz, pulsar Limpiar devuelve el texto al estado «ninguno». No exige modificar el catálogo ni crear una arquitectura nueva; practica la misma relación entre evento, campo y representación.

Una variante opcional consiste en estudiar qué ocurre al seleccionar un producto y añadir otro inmediatamente después. La selección debe continuar referida al producto anterior hasta que el usuario realice una nueva selección o limpie el estado, porque nuestros métodos no cambian `seleccion` automáticamente al añadir un objeto.

Diagnóstico de una interfaz que no cambia

Si el botón existe pero el texto nunca se actualiza, comprueba primero que la página declara `@rendermode InteractiveServer` y que la conexión interactiva está

establecida. Después verifica que activaste tanto el botón como el método C# correspondiente. Una línea Razor visible no garantiza que exista su método en el código compilado.

Si el proyecto no compila después de añadir el botón, revisa exactamente el nombre del método y la variable utilizada en su lambda. Si la lista muestra dos representaciones del mismo producto al añadir uno nuevo, comprueba que no has duplicado el bloque de lista mientras convertías la página. Si Añadir no hace nada con un nombre válido, asegúrate de haber activado `@bind` y el campo `nuevoNombre`, y de que el botón ejecuta `Agregar`.

Si el campo se vacía pero el producto no aparece, comprueba qué colección está representando el `@foreach`; debe ser la misma lista a la que se añadió el nuevo objeto. La depuración útil sigue la ruta **entrada → campo → método → colección → representación**.

Profundización opcional: cuándo se vuelve a representar un componente

Blazor suele programar una representación después de que un manejador de evento cambie estado. Eso explica por qué, en nuestros métodos sencillos, no necesitamos realizar una manipulación directa del DOM ni invocar `StateHasChanged` de forma explícita. Sin embargo, no deberíamos convertirlo en la afirmación de que cualquier modificación de un campo C# siempre actualiza la pantalla instantáneamente: los cambios que proceden de temporizadores, callbacks externos u otros contextos pueden requerir coordinarse con el contexto de representación.

No abordamos esas técnicas avanzadas en M03. La conclusión relevante es reconocer que el componente tiene un ciclo de representación administrado por el framework y que las operaciones ordinarias de `@onclick` y `@bind` participan en él.

Referencia: Microsoft Learn, Tratamiento de eventos y Enlace de datos en componentes.

Análisis de funcionamiento: tres cambios de estado de naturaleza distinta

P3.4 contiene tres operaciones que pueden confundirse si solo observamos que todas producen HTML actualizado. Seleccionar un producto cambia una **cadena** en el componente; escribir en un input cambia un **campo enlazado**; añadir un producto cambia el **contenido de una colección**. Blazor participa en representar los efectos de las tres operaciones, pero no es correcto afirmar que todas ejecutan exactamente el mismo tipo de mutación.

Cuando seleccionamos Lápiz, el método `Seleccionar(int id)` utiliza el identificador recibido y consulta la lista local para obtener el nombre. La información que aparece en pantalla procede del campo `seleccion`. La lista no se modifica, por lo que la cantidad de tarjetas sigue siendo tres. Si se observa un cambio de cantidad al seleccionar, debemos buscar otra operación no prevista o una variante de código accidentalmente activa.

Cuando escribimos en el campo asociado a `@bind="nuevoNombre"`, el navegador y el componente intercambian el valor según el evento del enlace. Esta modificación prepara la información que una operación posterior podría utilizar. Por sí sola no tiene motivo para alterar `productos`. Por ello el paso 3.4.3 se comprueba sin activar el botón de alta: necesitamos entender que disponer de una entrada enlazada no equivale a haber ejecutado una operación sobre el catálogo.

Cuando finalmente pulsamos Añadir, el método `Agregar()` lee el campo enlazado, decide si contiene una entrada aceptable, crea un objeto y lo añade a `productos`. Después limpia el campo. Esta secuencia enlaza dos cambios de estado: la colección contiene ahora un elemento más y el texto de entrada vuelve a estar vacío. Ambos deberían reflejarse en la representación. Si solo sucede uno, el diagnóstico debe separar el cambio de colección de la limpieza del campo.

Estado de entrada, validación mínima y límite del ejercicio

La condición `string.IsNullOrWhiteSpace(nuevoNombre)` responde a una necesidad puntual: evitar crear productos sin nombre útil. No pretende representar el conjunto de validaciones de un formulario empresarial. En M02 aprendimos

`Required`, `StringLength`, `Range` y `ModelState`, junto con la persistencia en SQL Server. Repetir ahora todo ese recorrido dentro de un componente Blazor diluiría el objetivo formativo de la sesión: **el alumno ya conoce los requisitos de datos; está aprendiendo un nuevo modelo de interfaz**.

Por ello el precio se fija en `5m` y el identificador se obtiene de un contador local. En este pequeño escenario no existe una estrategia de generación de identificadores apta para uso concurrente entre varios usuarios. Cada instancia de componente trabaja con su propia colección y su propio contador. La decisión es válida para el laboratorio, siempre que no presentemos el resultado como diseño definitivo de base de datos.

El alumno debe poder explicar qué comportamiento sería incorrecto: añadir un nombre vacío, crear dos objetos con la misma identidad dentro de esa lista, borrar por accidente otro objeto o declarar la operación completada solo porque el botón cambia de color. Las comprobaciones reales tienen que mirar la colección representada y el texto después del evento.

El papel del cierre de expresiones lambda

Dentro del `foreach` encontramos `@onclick="() => Seleccionar(producto.Id)"`. La lambda expresa qué trabajo debe ejecutar Blazor cuando llega el clic. No se está ejecutando `Seleccionar` durante la construcción del documento: se prepara una función que utiliza el identificador del elemento correspondiente. Esa distinción entre **construir el callback** y **ejecutar el callback** es una de las ideas esenciales de los frameworks de componentes.

Es útil comparar con una forma que no tendría el mismo significado:

```
@* Ejemplo de diagnóstico, NO sustituye el botón real:
   <button @onclick="Seleccionar(producto.Id)">Seleccionar</button>
*@
```

La variante intenta utilizar el resultado de ejecutar un método donde se espera un manejador, en lugar de proporcionar una función que pueda invocarse después. El ejemplo sirve para explicar un problema de tipos y momento de ejecución; el proyecto ya incluye la forma correcta, sin necesidad de introducir esta variante dentro del recorrido del alumno.

El renderizado posterior al evento no es una mutación del DOM escrita a mano

Una persona que viene de JavaScript puede buscar una instrucción equivalente a `document.getElementById("seleccion").textContent = ...`. No la encontrará en nuestro componente, porque declaramos qué contenido debe representar Razor en función de `seleccion`. Después del evento, Blazor actualiza la interfaz de acuerdo con el estado. El código del componente expresa un modelo declarativo: **qué debería verse para un valor determinado**, no una secuencia manual de operaciones sobre nodos HTML.

Esto facilita razonar sobre el mismo contenido desde distintas rutas de cambio. Si seleccionamos Cuaderno y después pulsamos Limpiar, ambos métodos modifican `seleccion` y la misma expresión Razor refleja el resultado. No necesitamos que cada método conozca el identificador DOM del párrafo. El elemento `id="seleccion"` existe en el laboratorio para facilitar la observación y las pruebas, no porque la aplicación lo necesite para mutar el texto.

La importancia de secuenciar eventos y binding

Considera un alumno que escribe «Regla» en el campo y pulsa Añadir inmediatamente. El enlace de datos del input y el evento del botón deben cooperar para que `Agregar` lea el valor actualizado. Por esa razón recomendamos comprobar primero la relación `@bind` con el campo y después activar el botón. Esta separación también ayuda si se produce un comportamiento extraño: podemos determinar si falla la entrada del dato o la operación que lo consume.

En componentes reales hay distintas opciones para los eventos de enlace, por ejemplo actualizar en cada pulsación de tecla con `@bind:event="oninput"`. No la adoptamos como nueva práctica obligatoria porque añadir variantes antes de comprender el enlace ordinario suele generar confusión. El alumno sí debe saber que el momento de actualización del valor depende de la configuración del binding.

Persistencia aparente y vida de la instancia

Tras pulsar Añadir, la tarjeta se encuentra en la lista local durante la vida de esa instancia. Si la persona vuelve a la página mediante una navegación que crea otra instancia, el componente puede inicializar la lista otra vez. La ausencia de persistencia no debe diagnosticarse como un fallo de `@onclick`. Es la consecuencia de una arquitectura explícita: no hay ninguna operación de guardar y no existe un repositorio de datos externo conectado.

Esta distinción protege al alumno de un falso paralelismo con M02. Allí un POST podía llegar a `SaveChangesAsync` y una nueva consulta SQL confirmaba la fila. Aquí la prueba positiva consiste en observar el cuarto componente dentro de la sesión actual. Son evidencias de operaciones diferentes; no debemos aplicar el mismo criterio de persistencia a mecanismos que no la ofrecen.

Un caso de transferencia: la selección no debería perderse al añadir un producto

En nuestra implementación, `Agregar()` modifica la lista y limpia `nuevoNombre`, pero no modifica `seleccion`. Si el usuario había seleccionado Lápiz, esa selección puede mantenerse cuando se añade Regla. Sería incorrecto asumir que toda actualización de la colección reinicia automáticamente todos los campos del componente. Un componente mantiene varios elementos de estado independientes y cada método determina cuáles modifica.

Esta observación prepara P3.5, donde el padre decidirá qué hacer con una selección comunicada desde una tarjeta y cómo resetearla cuando se quite un producto. La lógica de estado reside en el componente que lo posee, y los eventos deben comunicar intenciones claras para que ese componente pueda aplicar sus reglas.

Preguntas de diagnóstico con respuesta causal

Si pulsamos Seleccionar y aparece otro nombre, ¿qué revisamos? El Id capturado por la lambda y el objeto que `First` encuentra en la lista. Si escribimos un nombre y se añade un producto vacío, ¿qué revisamos? El campo enlazado, el evento del input y la condición de `Agregar`. Si la lista se actualiza pero el input no se limpia, ¿qué revisamos? La asignación a

`string.Empty` y la representación del binding. Si el texto nunca cambia pese a compilar, ¿qué revisamos? Primero el modo interactivo y la conexión, después el manejador.

Estos diagnósticos responden a síntomas diferentes. Aprender a distinguirlos evita arreglos arbitrarios que podrían ocultar la causa. No necesitamos incorporar logs empresariales ni herramientas de estado global; la observación del código, los puntos de interrupción y la respuesta visible bastan para los objetivos de este módulo.

Un editor real en memoria: identidad, record y representación

Hasta ahora P3.4 ha demostrado dos operaciones: seleccionar un producto sin cambiar la colección y añadir otro elemento a esa colección. El temario pide además un **editor simple** que permita cambiar un dato del producto seleccionado y observar el resultado en pantalla. El paso 3.4.5 completa esta necesidad sin repetir la infraestructura de formularios y EF Core de M02.

Editar correctamente exige responder tres preguntas. ¿Qué producto queremos modificar? ¿Qué valor nuevo ha escrito el usuario? ¿Qué componente es responsable de actualizar la colección? La primera pregunta no debería resolverse utilizando un nombre, porque pueden existir nombres repetidos. La segunda necesita un campo enlazado cuyo contenido pueda diferir temporalmente del dato original. La tercera corresponde al componente propietario de la lista.

Conservar la identidad del producto

El método de selección inicial guarda el nombre que debe mostrarse en la cabecera. Eso es suficiente para una demostración visual, pero no para identificar sin ambigüedad un producto durante una edición. El editor introduce dos campos nuevos:

```
private int? idEnEdicion;
private string nombreEditado = string.Empty;
```

La nulabilidad del identificador expresa que al abrir la página aún no se ha elegido ningún producto. El texto `nombreEditado` almacena la propuesta que se

introduce en el control. La propiedad `seleccion` continúa encargándose del resumen visible. Separar estos tres conceptos evita usar una cadena como identificador o aplicar cambios a un elemento distinto cuando dos productos tienen el mismo nombre.

Al pulsar Seleccionar en una tarjeta, el nuevo manejador inicializa el editor:

```
private void SeleccionarParaEditar(int id)
{
    var producto = productos.First(p => p.Id == id);
    idEnEdicion = id;
    seleccion = producto.Nombre;
    nombreEditado = producto.Nombre;
}
```

Observa las asignaciones por separado. El método recuerda la identidad, actualiza lo que se comunica al usuario como selección y rellena el campo que posteriormente podrá modificar. En ese momento todavía **no** ha alterado los datos del catálogo. La operación de selección y la operación de guardado siguen siendo independientes.

El botón que ejecuta ese método utiliza una lambda como la selección anterior, pero cambia el manejador:

```
<button @onclick="() => SeleccionarParaEditar(producto.Id)">
    Seleccionar
</button>
```

En el proyecto didáctico el botón aparece como una variante preparada para **sustituir** el botón de P3.4.2, no para acumular dos botones de selección dentro de cada tarjeta. Esta decisión conserva la continuidad de los pasos: podemos comenzar por una selección simple y evolucionar hacia la selección que prepara una edición.

La diferencia entre escribir un valor y aplicarlo

El campo de edición tiene un binding independiente:

```
<label>Editar nombre:
    <input id="editar-nombre" @bind="nombreEditado" />
</label>
```

```
<button id="guardar-cambio" @onclick="GuardarCambio">
    Guardar nombre
</button>
```

Supongamos que hemos seleccionado Lápiz y que el campo muestra «Lápiz». El usuario escribe «Lápiz rojo». Esa escritura prepara un valor nuevo que se comunica al componente mediante `@bind`. La lista de productos puede seguir mostrando el nombre original hasta que se pulse Guardar nombre. Esta separación hace visible una distinción habitual de la programación de interfaces: **el valor de edición no es necesariamente el valor ya confirmado**.

En MVC obtuvimos un comportamiento comparable mediante un formulario y una operación POST. Aquí el mecanismo no pasa por una acción MVC ni por `ModelState`: los eventos de un componente interactivo coordinan campos locales y representación. Que la idea de «confirmar un cambio» sea común no implica que el flujo de ejecución sea idéntico.

Sustituir un record conservando Id y precio

Nuestro modelo Producto es un `record`. En lugar de crear un objeto nuevo con un identificador distinto o intentar tratar sus propiedades posicionales como un objeto mutable cualquiera, utilizamos una expresión `with` para conservar las propiedades no modificadas:

```
private void GuardarCambio()
{
    if (idEnEdicion is null ||
        string.IsNullOrWhiteSpace(nombreEditado))
        return;

    var indice = productos.FindIndex(
        p => p.Id == idEnEdicion.Value);

    if (indice < 0)
        return;

    productos[indice] = productos[indice] with
    {
        Nombre = nombreEditado.Trim()
    };
```

```
    seleccion = productos[indice].Nombre;
}
```

La primera condición rechaza una edición sin selección y una entrada vacía o compuesta por espacios. El `FindIndex` localiza el lugar que ocupa el producto de ese Id dentro de la colección. Si no existe, retorna sin tocar otro producto. La expresión `with` construye el nuevo valor que conserva el Id y el precio mientras cambia el nombre, y la asignación sustituye el elemento dentro de la lista.

No debe confundirse esta operación con `productos.Add`: añadir crea un elemento más; editar sustituye el valor de un elemento existente. Al final de la operación debe haber tres productos, no cuatro. El Id del producto editado seguirá siendo 2 si habíamos seleccionado Lápiz. Su precio también debe mantenerse, porque la expresión `with` solo especifica la propiedad Nombre.

La última asignación actualiza `seleccion` con el nuevo nombre. Si el usuario acaba de cambiar Lápiz por «Lápiz rojo», mantener la frase «Seleccionado: Lápiz» sería un estado visual incoherente. El componente que posee la colección es también responsable de sincronizar los otros campos de presentación que dependen de ese cambio.

El recorrido positivo y sus comprobaciones

El alumno puede reconstruir esta edición sin salir de `/productos`. Primero ve tres tarjetas. Después pulsa Seleccionar en Lápiz y comprueba que el editor se rellena con su nombre. Escribe «Lápiz rojo» y pulsa Guardar nombre. La tarjeta con Id 2 presenta el nombre actualizado y conserva el precio. El número de tarjetas sigue siendo tres. La cabecera de selección también muestra el valor nuevo.

Cada observación sirve para comprobar una regla. La coincidencia de Id prueba que se modificó el elemento correcto. La cantidad constante prueba que no se añadió otro objeto accidentalmente. El precio conservado prueba que la expresión `with` mantuvo los valores ajenos al cambio. La cabecera actualizada prueba que el componente mantuvo una representación coherente.

En una prueba negativa, el usuario introduce únicamente espacios y pulsa Guardar. El método debe rechazar el cambio, por lo que la tarjeta conserva el nombre anterior. Otra prueba negativa consiste en pulsar Guardar sin seleccionar antes un producto. Como `idEnEdicion` sigue siendo null, no se modifica una tarjeta arbitraria. Estas situaciones se pueden estudiar con C# y la interfaz existente, sin construir una infraestructura de errores remotos.

Qué enseña exactamente este editor

Aunque parezca una operación de edición de un CRUD, **no es el mismo ejercicio que M02**. Allí se aprendió a procesar formularios, validar ModelState y persistir mediante EF Core. Aquí la competencia central es relacionar identidad de un componente, binding, evento de guardado, modificación de una colección y nueva representación. El cambio existe en memoria y puede desaparecer cuando el circuito se reinicia o el componente vuelve a crearse.

Esta delimitación ayuda a evitar un falso aprendizaje: no deberíamos describir la operación como un UPDATE SQL, porque el proyecto no contiene ninguna llamada que lo realice. La explicación debe seguir lo que realmente hace el código, incluso cuando los nombres de los botones se parezcan a los de la aplicación MVC anterior.

Un diagnóstico que parte del síntoma y no del framework

Si la selección visual cambia pero el campo de edición no se rellena, comprueba que se utiliza `SeleccionarParaEditar` y no el manejador anterior. Si al guardar se crea una cuarta tarjeta, revisa que no hayas sustituido `productos[indice] = ...` por `productos.Add(...)`. Si se modifica la tarjeta equivocada, inspecciona `idEnEdicion` y el predicado de `FindIndex`. Si el nombre cambia pero el precio también, investiga cómo construiste el record actualizado. Si el botón no responde, comprueba que sus fragmentos Razor y C# se activaron conjuntamente y que la página sigue siendo interactiva.

Estas preguntas preparan una destreza transferible: un editor de componentes no se verifica únicamente comprobando que su código compila. Debe conservar una identidad, aplicar el cambio correcto, rechazar una

entrada inválida y producir una representación coherente. P3.4.5 permite observar esas propiedades con muy pocos conceptos nuevos.

3.5. Comunicación padre–hijo: parámetros y EventCallback

Cuando el botón está dentro del hijo, pero el dato pertenece al padre

En 3.3 aprendimos a pasar un producto a una tarjeta reutilizable. En 3.4 gestionamos selección y altas desde una página que contiene directamente sus botones. Ahora aparece una situación muy habitual: queremos que el botón Seleccionar forme parte de `TarjetaProducto.razor`, porque cada tarjeta debe mostrar sus propias acciones, pero la selección actual y la lista de productos siguen perteneciendo a `Productos.razor`. **¿Cómo puede el hijo comunicar una intención sin asumir la propiedad de la lista del padre?**

Una respuesta incorrecta sería intentar que el hijo busque y modifique de forma oculta la colección del padre, o almacenar una segunda copia independiente que se desincronice. Blazor ofrece `EventCallback<TValue>`, un contrato de salida del componente que permite notificar al padre y transportar un valor tipado.

En P3.5 la tarjeta comunica un Id de producto. El padre recibe ese Id, busca el producto correspondiente y actualiza el campo `seleccion`. El componente hijo no decide cómo se almacena el estado de selección ni qué acciones adicionales ejecuta el padre. Esa separación mantiene el componente reutilizable y fácil de comprender.

Parámetros de entrada y callbacks de salida

En `TarjetaProducto.razor` conservamos:

```
[Parameter, EditorRequired]
public Producto Producto { get; set; } = default!;
```

Y en el paso 3.5.2 activamos:

```
[Parameter]
public EventCallback<int> AlSeleccionar { get; set; }
```

```
private Task NotificarSeleccion()
    => AlSeleccionar.InvokeAsync(Producto.Id);
```

Ambas propiedades llevan `[Parameter]`, pero representan contratos diferentes. `Producto` contiene el **dato que el padre proporciona** al hijo. `AlSeleccionar` contiene el callback al que el hijo puede notificar una decisión del usuario. `EventCallback<int>` declara qué tipo de dato se enviará: en nuestro caso, el identificador numérico.

El método `NotificarSeleccion` devuelve un `Task`, adecuado para representar el trabajo asíncrono que pueda realizar el callback. En este ejemplo la actualización del padre es sencilla, pero la firma deja clara la relación con `InvokeAsync`. No añadimos una llamada HTTP solo para comunicar dos componentes que viven dentro de la misma aplicación.

El botón que emite la notificación

La tarjeta contiene el bloque activable:

```
<button @onclick="NotificarSeleccion">Seleccionar</button>
```

Cuando se pulsa, la tarjeta invoca su método y el callback transporta `Producto.Id` hacia el padre. La tarjeta no necesita conocer el nombre del método del padre: únicamente conoce la propiedad `AlSeleccionar`. Esta es la diferencia entre un acoplamiento rígido y un contrato de evento reutilizable.

El padre, en `Productos.razor`, activa la variante:

```
<TarjetaProducto Producto="producto"
                 AlSeleccionar="Seleccionar" />
```

Y define:

```
private void Seleccionar(int id)
{
    seleccion = productos.First(p => p.Id == id).Nombre;
}
```

El hijo notifica el Id; el padre decide cómo actualizar su propia selección. Al finalizar el evento, la representación del padre muestra el nuevo valor de `seleccion`. La comunicación es observable: al seleccionar Lápiz, aparece el texto «Seleccionado: Lápiz» fuera de la tarjeta.

Dirección de la información y propiedad del estado

El flujo completo ahora tiene dos direcciones complementarias. La información del producto va del padre al hijo mediante un parámetro. El evento de selección va del hijo al padre mediante `EventCallback<int>`. **No es una sincronización automática bidireccional de cualquier propiedad**; son dos contratos expresos.

Esta separación es importante al diseñar componentes. Podemos reutilizar la tarjeta en otra página que responda de forma diferente a una selección, porque el hijo solo informa del hecho. El padre puede cambiar un texto, navegar, abrir una ventana o ejecutar una acción que diseñemos más adelante; nada de eso está codificado en la tarjeta de P3.5.

La práctica evita convertir EventCallback en un mecanismo general de mensajería entre partes arbitrarias de la aplicación. Se utiliza para el escenario al que mejor responde: comunicar la intención de un componente hijo al componente que lo contiene.

Por qué la variante debe sustituir a la anterior

El estado inicial de P3.5 contiene una línea activa:

```
<TarjetaProducto Producto="producto" />
```

Y otra comentada con el callback. La práctica indica **sustituir** la primera por la segunda después de activar las propiedades y métodos del hijo y del padre. No basta con descomentar ambas, porque eso produciría dos tarjetas por producto. La regla docente es que cada transición transforma un bloque conocido en otro estado verificable, no que el alumno acumule todas las soluciones una debajo de otra.

Si se activa el parámetro `AlSeleccionar` desde el padre pero el componente hijo todavía no lo declara, el compilador puede advertir que el componente no reconoce ese parámetro o que el código no es válido. No es un problema de enrutamiento. La dependencia es directa: la firma del hijo y la invocación desde el padre deben evolucionar conjuntamente.

Reto: solicitar la eliminación de un producto

El reto 3.5 incorpora una segunda intención: «Quitar». La tarjeta emite el Id del producto mediante:

```
[Parameter]
public EventCallback<int> AlQuitar { get; set; }

private Task NotificarQuitar()
    => AlQuitar.InvokeAsync(Producto.Id);
```

Y el padre dispone del método:

```
private void Quitar(int id)
{
    productos.RemoveAll(p => p.Id == id);
    seleccion = "ninguno";
}
```

En el estado del reto, el padre pasa ambos callbacks:

```
<TarjetaProducto Producto="producto"
                 AlSeleccionar="Seleccionar"
                 AlQuitar="Quitar" />
```

La tarjeta no borra la colección directamente: comunica una solicitud tipada. Es el padre quien modifica su lista, provoca el nuevo total visible y limpia la selección si corresponde. En nuestra práctica el resultado es local y temporal; no existe petición HTTP DELETE ni operación de EF Core. Precisamente por eso podemos estudiar el patrón de comunicación sin añadir otra capa.

Pruebas positivas y negativas

La prueba positiva selecciona un elemento conocido y observa el texto del padre. Después puede quitarlo y comprobar que disminuye el contador de productos, que su tarjeta ya no está representada y que la selección vuelve a «ninguno». Una prueba de consistencia consiste en intentar localizar visualmente el producto eliminado: no debería seguir apareciendo dentro de la lista actual.

Al refrescar de forma que se crea una instancia nueva del componente, los productos iniciales vuelven a mostrarse. Este comportamiento no es una regresión del callback; deriva de que la lista continúa siendo memoria local. No confundimos quitar un elemento de una colección en un circuito con eliminar una entidad persistente del CRUD MVC de M02.

Diagnóstico de errores de comunicación

Si el botón aparece pero el padre no cambia, comprueba que el hijo ejecuta `InvokeAsync` y que el atributo del componente padre enlaza el método correcto. Después verifica que ambos lados utilizan un tipo compatible, `int` en este laboratorio. Si se actualiza una selección equivocada, mira qué Id emite la tarjeta y qué elemento busca el padre.

Si aparecen seis tarjetas cuando deberían existir tres, el problema probablemente está en haber dejado activas dos variantes del `foreach`; no es evidencia de que `EventCallback` duplique el contenido por sí mismo. Si el reto elimina todos los productos por error, revisa el predicado de `RemoveAll`; debe comparar el Id que recibió, no un valor constante.

Para depurar este patrón resulta útil escribir un pequeño recorrido: clic en hijo → `NotificarSeleccion` → `InvokeAsync(Producto.Id)` → `Seleccionar(id)` del padre → cambio de `seleccion` → representación. Cada elemento corresponde a un archivo o método observable.

Profundización opcional: eventos tipados y contratos pequeños

`EventCallback<TValue>` resulta especialmente útil porque el tipo comunicado es explícito. Nuestro `int` no transporta todos los datos de un producto; solo la

identidad necesaria para que el padre decida qué hacer. Podríamos usar `EventCallback<Producto>` en otras situaciones, pero no deberíamos elegirlo por inercia si un Id es suficiente.

Un contrato pequeño reduce la información compartida y mantiene claras las responsabilidades. Esta idea es transferible a otros frameworks de componentes, aunque las herramientas sintácticas sean diferentes. No necesitamos introducir bibliotecas de estado global, mediadores, arquitecturas empresariales o patrones avanzados para apreciar su valor: basta con observar que un hijo reutilizable no modifica silenciosamente el estado que otro componente posee.

Referencia: Microsoft Learn, Comunicación y EventCallback.

Profundización: un protocolo pequeño para comunicar intenciones

La relación padre–hijo de Blazor puede comprenderse mediante un contrato de dos direcciones. En la entrada, la propiedad `Producto` contiene el objeto que la tarjeta representará. En la salida, `AlSeleccionar` o `AlQuitar` contienen callbacks que notifican intenciones. En ninguno de los dos casos el componente hijo necesita recibir una referencia directa a la colección mutable del padre.

Este diseño reduce el acoplamiento. Si mañana la página sustituye la lista local por resultados de un servicio, la tarjeta puede continuar recibiendo un `Producto` y emitiendo un Id. El padre podría interpretar `AlQuitar` de otra manera, por ejemplo solicitar confirmación o realizar una operación contra una API futura, sin que la tarjeta necesite conocer todos esos detalles.

Tres formas diferentes de compartir información

Podemos comparar tres diseños. El primero copia una propiedad a un hijo por parámetro y deja que el hijo la muestre. Es lo que construimos en P3.3. El segundo coloca un botón en el padre y modifica directamente su estado; ese es el modelo de P3.4. El tercero coloca el botón en el hijo, que debe comunicar la intención al padre; ese es el modelo de P3.5. Los tres pueden ser correctos según la responsabilidad que deseamos modelar.

El error aparece cuando confundimos los contratos. Pasar `Producto` al hijo no le concede automáticamente autoridad sobre toda la colección. Dibujar un botón dentro de la tarjeta tampoco significa que el padre vaya a enterarse de la operación sin haber conectado un callback. Necesitamos una relación explícita, y `EventCallback<int>` ofrece precisamente esa posibilidad.

Qué garantiza el tipo del callback y qué no

La declaración `EventCallback<int>` establece que el callback trabaja con un valor entero. Este tipo ayuda al compilador y a las herramientas a identificar combinaciones incorrectas. No garantiza, sin embargo, que el entero corresponda a un producto realmente existente o que la persona tenga autorización para quitarlo en una aplicación de producción. Son preguntas de validación y seguridad distintas.

En el laboratorio, el Id procede del objeto que la tarjeta está representando. El padre sabe qué lista utiliza y puede buscarlo. Si el Id no existe en el contexto del padre, la implementación de `First` en la selección podría generar una excepción; en aplicaciones reales convendría decidir explícitamente qué hacer ante ese caso. No añadimos aquí una capa avanzada de manejo de estados concurrentes porque nuestra colección y nuestra secuencia de interacción son pequeñas y controladas.

El reto de eliminación utiliza `RemoveAll(p => p.Id == id)`, que produce una nueva lista lógica con menos elementos, pero no modifica datos almacenados en SQL. Debemos distinguir un resultado visible del componente de un efecto persistente: el contrato de callback solo expresa comunicación entre componentes; la semántica de negocio depende del método del padre.

Por qué se utiliza InvokeAsync

`AlSeleccionar.InvokeAsync(Producto.Id)` es la forma explícita de notificar un callback. Devuelve una tarea que puede representar una operación asíncrona. El hijo no necesita conocer si el padre implementó un método sincrónico pequeño o si más adelante el tratamiento será asíncrono. Esta decisión deja abierto un patrón de evolución sin obligar a introducir una API durante M03.

No debemos sustituir el callback por una llamada directa inventada como `Padre.Seleccionar(...)`, porque el componente hijo no dispone de una referencia estructurada al objeto padre ni debería depender de una clase concreta para presentar una tarjeta. La propiedad de callback es un contrato público del componente, igual que su propiedad de datos, y se conecta en la etiqueta del padre.

Transferencia desde la vista parcial MVC

Recordemos la vista parcial del catálogo MVC: recibía un modelo y generaba un fragmento de HTML. Si necesitaba realizar una acción, normalmente representaba un enlace o formulario que conduciría a otro endpoint HTTP. En nuestra tarjeta Blazor, un botón puede emitir un callback que cambia inmediatamente el estado del padre sin navegar a otra URL. Esa es una capacidad relevante del modelo interactivo.

No significa que las vistas parciales sean «malas» ni que los componentes deban reemplazarlas en todas las aplicaciones. Es una diferencia de comportamiento disponible y de responsabilidades. El alumno debería ser capaz de identificar cuál conviene cuando la operación principal consiste en navegar hacia otro recurso y cuál resulta expresiva cuando queremos actualizar una selección local sin cambiar de pantalla.

Caso de diagnóstico: tarjeta duplicada después de activar callbacks

En el proyecto P3.5 se entregan variantes completas del componente padre: una sin callback, otra con `AlSeleccionar` y una tercera con `AlSeleccionar` y `AlQuitar`. Si el alumno descomenta las variantes sin sustituir la línea activa, cada iteración del `foreach` produce dos o tres tarjetas. La cantidad de elementos HTML aumenta, pero no necesariamente la longitud de la colección `productos`. Esto es un error de **composición**, no un error de `EventCallback`.

El diagnóstico correcto empieza por contar las instancias de `TarjetaProducto` dentro de la iteración. Después se comprueba que cada variante tenga declarados sus parámetros en el hijo y sus métodos correspondientes en el padre. Solo al completar ambos lados tiene sentido probar la navegación o la interacción.

Caso de diagnóstico: la tarjeta se quita, pero el texto sigue seleccionado

Imaginemos que modificamos `Quitar(int id)` para eliminar el producto, pero olvidamos limpiar `seleccion`. Si eliminamos Lápiz cuando estaba seleccionado, podríamos continuar leyendo «Seleccionado: Lápiz» mientras la tarjeta ya no existe. No hay un problema de transmisión del evento: el callback llegó al padre y `RemoveAll` funcionó. El problema es que **el padre no actualizó otro campo del estado que debía cambiar de forma coherente**.

Por eso el reto incluye `seleccion = "ninguno"`. La solución expresa una pequeña regla de consistencia local. El hijo no debe decidir cómo actualizar ese estado; corresponde al padre porque conoce tanto la selección como la lista. Esa decisión ejemplifica el beneficio de mantener la propiedad del estado en un único lugar claro.

Reutilización conceptual en M04

Cuando consumamos una API, un botón de tarjeta podría notificar un Id al padre y el padre podría llamar a un servicio HTTP. Pero no es necesario que el componente conozca el protocolo de comunicación para seguir siendo reutilizable. Esta separación evita trasladar los detalles de transporte a cada botón de cada tarjeta.

El objetivo de P3.5 termina antes de esa frontera: el alumno debe demostrar un callback real, seleccionar y quitar elementos, y explicar qué parte del estado cambió. Una descripción que solo diga «el botón manda un evento» resulta insuficiente si no identifica el Id, el método receptor y el efecto visible en la lista.

3.6. OnInitializedAsync y ciclo de vida esencial de un componente

El siguiente problema: los datos no siempre están listos al representar

Hasta 3.5 escribimos la lista de productos directamente dentro del componente. Esto es suficiente para aprender composición y eventos, pero en una aplicación real los datos pueden proceder de otro servicio y tardar en obtenerse. Antes de introducir una API REST en M04, necesitamos entender **cuándo** puede un componente cargar información y **cómo** debe representarse la pantalla mientras esa información todavía no está disponible.

En P3.6 introducimos un servicio **en memoria**, no HTTP. El servicio devuelve tres productos después de una espera controlada; esto permite observar el estado «Cargando productos...» y después «Lista preparada». Se trata de una simulación didáctica de una operación asíncrona: el objetivo no es enseñar `Task.Delay` como patrón de acceso a datos, sino ofrecer una transición temporal comprensible.

Separar adquisición de datos y presentación

La interfaz del servicio es:

```
public interface IProductosMemoriaService
{
    Task<IReadOnlyList<Producto>> ObtenerAsync();
}
```

La implementación `ProductosMemoriaService` construye productos en memoria. Su método `ObtenerAsync` utiliza `await Task.Delay(700)` antes de devolver la lista. No contiene `HttpClient`, una URL de API, un DbContext ni consultas SQL. Es importante que el estudiante pueda abrir el servicio y comprobarlo por sí mismo.

La página Productos se concentra en decidir qué se representa durante la espera y al finalizar. El servicio se ocupa de proporcionar los datos. Este

diseño permite que en M04 podamos introducir HTTP sin convertir la página en una colección de detalles de transporte.

Registrar el servicio mediante DI

El proyecto 3.6 incorpora en `Program.cs`:

```
builder.Services.AddScoped<
    IProductosMemoriaService, ProductosMemoriaService>();
```

El contenedor de ASP.NET Core sabe qué implementación proporcionar cuando se solicita la interfaz. Desde el componente podemos pedirla con:

```
@inject IProductosMemoriaService Catalogo
```

No instanciamos el servicio mediante `new` dentro de cada página. Además de facilitar las pruebas, esto enseña una idea que el alumno ya conoce de Web API: la inyección de dependencias es una infraestructura de ASP.NET Core que también puede utilizarse desde componentes Blazor.

En Interactive Server, el alcance de servicios `Scoped` tiene particularidades relacionadas con el circuito interactivo. No debemos suponer sin más que se crea una instancia nueva por cada clic ni equipararlo exactamente a la vida de una solicitud HTTP MVC. Nuestro servicio no mantiene datos mutables entre llamadas, de modo que no depende de esas sutilezas; nos basta con identificar que el contenedor lo suministra al componente.

OnInitializedAsync como lugar de carga inicial

El código preparado en la página incluye:

```
protected override async Task OnInitializedAsync()
{
    estado = "Cargando productos...";
    productos = await Catalogo.ObtenerAsync();
    cargando = false;
    estado = "Lista preparada";
}
```

`OnInitializedAsync` es un método del ciclo de vida que Blazor invoca para inicializar el componente. En él cargamos los datos necesarios para su primera experiencia. El código coloca primero el mensaje de espera, obtiene la lista y marca el final. La transición también depende de que la página pueda representarse mientras espera, por lo que mantenemos un estado válido antes del `await`.

No necesitamos llamar a `OnInitializedAsync` desde un botón ni hacerlo público para el navegador: lo invoca el framework. La sobreescritura utiliza `protected override` porque pertenece al contrato de `ComponentBase`. Esto lo diferencia de `OnGet` de Razor Pages, cuya ejecución responde a un verbo HTTP. Los nombres pueden sonar parecidos, pero pertenecen a modelos de programación distintos.

Estados visibles del componente

La página mantiene:

```
private IReadOnlyList<Producto> productos =
    Array.Empty<Producto>();
private bool cargando = true;
private string estado = "Servicio pendiente de activar";
```

Y en su marcado:

```
<p id="estado-carga">@estado</p>

@if (!cargando && productos.Count > 0)
{
    <div class="tarjetas" id="tarjetas-productos">
        @foreach (var producto in productos)
        {
            <TarjetaProducto Producto="producto" />
        }
    </div>
}
```

Hay una decisión importante: la lista comienza vacía y la condición de la vista evita intentar representar objetos que todavía no se han cargado. La bandera `cargando` separa «todavía espero el resultado» de «ya terminó la carga». El

mensaje `estado` ofrece al alumno una representación textual adicional para observar la transición. No son dos bases de datos ni dos servicios; son campos de la misma instancia de componente.

Cuando el servicio termina, `productos` recibe la colección y `cargando` se convierte en `false`. La condición permite mostrar tres tarjetas. La salida positiva no es solo que se haya compilado un método asíncrono: el navegador debe llegar a representar «Lista preparada» y los tres productos esperados.

Prerenderizado y por qué esta página lo desactiva en el laboratorio

En Blazor Web App con Interactive Server, los componentes pueden prerenderizarse en el servidor antes de establecer el circuito interactivo. En ese caso, `OnInitializedAsync` puede ejecutarse durante el prerenderizado y otra vez al inicializar el componente interactivo. Es un comportamiento importante de .NET 8 y puede sorprender a quien proviene de MVC, donde suele asociar la carga de una vista con una única petición.

El proyecto 3.6 hace explícita una elección didáctica:

```
@rendermode @(new InteractiveServerRenderMode(prerender: false))
```

Desactivamos el prerenderizado **en esa página** para observar una transición sencilla y evitar que la simulación de espera se ejecute dos veces por dos fases de representación. No presentamos esta configuración como universalmente superior para producción. El prerenderizado tiene beneficios de experiencia inicial y accesibilidad que debemos considerar en una aplicación real.

La alternativa exigiría estudiar persistencia de estado de prerenderizado o estrategias para evitar duplicación de operaciones costosas. No las desarrollamos en un módulo introductorio; sí explicamos por qué, si alguien cambia el modo de renderizado, puede observar dos llamadas al servicio y no una sola. El objetivo es enseñar una decisión consciente, no ocultar el comportamiento del framework.

La diferencia entre estado vacío y error

Un catálogo sin elementos no es necesariamente un fallo de red ni una excepción. La respuesta de un servicio puede ser una lista vacía perfectamente válida. Por eso el reto 3.6 prepara una condición adicional:

```
@if (!cargando && productos.Count == 0)
{
    <p id="sin-productos">No hay productos para mostrar.</p>
}
```

Para probarla, el alumno cambia temporalmente el servicio para que devuelva una lista vacía. Cuando termina la espera, debe aparecer el mensaje de ausencia de datos. El estado de «cargando» ya terminó: no debemos dejar el indicador activo indefinidamente porque el resultado contenga cero elementos.

En M04 añadiremos fallos HTTP y sus correspondientes estados de error. En este punto no es necesario diseñar una jerarquía general de excepciones ni simular caídas de red. Sí debe quedar clara la distinción conceptual entre «esperando», «lista cargada», «lista vacía» y «operación fallida».

Qué hacer cuando OnInitializedAsync no termina como esperamos

Si la pantalla permanece en «Servicio pendiente de activar», comprueba que el método preparado en `Productos.razor` está realmente activo y que la aplicación se ha recompilado. Si aparece una excepción indicando que no se puede proporcionar `IProductosMemoriaService`, revisa la inscripción en `Program.cs` y el `@inject` de la página. Si los datos aparecen antes de que puedas observar la espera, confirma que el proyecto corresponde a P3.6 y que el modo de renderizado mantiene la opción de prerenderizado indicada.

Si el contador de productos no coincide con la lista que devuelve el servicio, inspecciona dónde asignas `productos` y qué colección representa el `@foreach`. Si la lista está vacía al finalizar, el mensaje correcto del reto es una condición de ausencia de datos, no una página rota. Si aparece repetidamente

«Cargando...», hay que investigar si `ObtenerAsync` termina y si el componente actualiza `cargando` después del `await`.

El razonamiento diagnóstico sigue un camino claro: inyección → inicialización → llamada asíncrona → retorno de datos → actualización de campos → nueva representación. Cada etapa tiene un lugar concreto en el código y un efecto observable.

Conexión con el siguiente módulo

En M04 sustituiremos parte del mecanismo de obtención de datos por llamadas a una API auxiliar. La interfaz seguirá teniendo componentes, eventos y estados de espera, pero habrá una nueva frontera: HTTP, serialización JSON, códigos de respuesta y errores de comunicación. Haber separado el servicio en 3.6 nos permite estudiar la nueva frontera sin empezar otra vez desde cero a construir la página.

Esto también explica por qué M03 no incluye una API. Si introdujéramos ahora una llamada remota junto con el ciclo de vida, los callbacks y el binding, sería difícil saber qué concepto causa cada comportamiento. Primero comprobamos que los componentes funcionan con datos controlados en memoria; después añadiremos transporte y manejo de errores.

Cierre conceptual: seis puntos que forman un único recorrido

Podemos narrar la jornada como una cadena: primero comprobamos que la interactividad requiere una configuración efectiva y la distinguimos del SSR estático; después localizamos los componentes de página y su navegación; extraemos una tarjeta reutilizable; utilizamos eventos y enlace de datos para modificar estado; trasladamos el botón a una tarjeta que notifica al padre; y finalmente cargamos los productos desde un servicio asíncrono mediante el ciclo de vida.

Cada paso reutiliza el lenguaje conceptual anterior. El alumno no necesita memorizar veinte directivas desconectadas; debe ser capaz de seguir una interacción desde el usuario hasta el método C# y hasta la representación

actualizada. Cuando pueda hacerlo para selección, eliminación de una tarjeta y carga asíncrona, tendrá la base necesaria para consumir una API en M04.

La demostración termina con un catálogo local y temporal, no con un CRUD persistente. M02 ya enseñó la persistencia; M03 enseña la composición y el comportamiento interactivo de la interfaz. Mantener separadas esas responsabilidades es parte del diseño del curso, no una carencia del proyecto.

Referencia: Microsoft Learn, Ciclo de vida de componentes Razor y Inyección de dependencias en Blazor.

Ampliación aplicada: secuencia de vida, representación y errores de carga

El ciclo de vida de componentes no debe aprenderse como una colección de nombres aislados. Cada método tiene sentido porque el framework necesita crear, inicializar, proporcionar parámetros, representar y eventualmente descartar componentes. En P3.6 nos concentramos en `OnInitializedAsync`, que permite iniciar una operación de obtención de datos cuando se prepara la instancia. El alumno ya conoce `async` y `await` de C#, de modo que la novedad no está en escribir una tarea, sino en entender **cuándo la invoca el framework** y cómo se relaciona con la representación.

Qué debe contener un estado representable antes del await

Cuando un método asíncrono inicia una operación que todavía no ha terminado, el componente puede necesitar producir una representación provisional. Por ello el proyecto define una lista vacía, la bandera `cargando` y el mensaje `estado` antes de obtener datos. Estos valores permiten que la página tenga una representación coherente tanto al empezar como después de la espera.

Una implementación que presupusiera que `productos` siempre está disponible con tres elementos podría intentar acceder a una posición inexistente antes de completar la carga. Nuestro marcado no lo hace: comprueba `!cargando && productos.Count > 0` antes de dibujar tarjetas. Esta condición explica por qué la pantalla inicial no debe interpretarse como una excepción, sino como un estado esperado.

El servicio utiliza `await Task.Delay(700)` para simular latencia. En una aplicación real podría existir una consulta a otra API, un acceso a almacenamiento o una operación de red. La espera artificial facilita la observación, pero no conviene copiarla como si fuera una recomendación para producir sistemas más lentos. El contenido formativo está en la transición y en la separación de responsabilidades, no en la duración exacta del retardo.

Una fuente de datos separada de su representación

`IProductosMemoriaService` establece un contrato: obtener una lista de productos de manera asíncrona. La página usa ese contrato mediante `@inject`. La implementación decide dónde construir la lista. Esta separación tiene un beneficio concreto: la página puede centrarse en los estados de presentación y la tarjeta puede seguir representando un objeto, sin saber de qué manera se obtuvo.

La idea se relaciona con la inyección de dependencias de Web API. En ambos casos, el contenedor registra una implementación y las clases o componentes pueden solicitar su contrato. La diferencia importante para Blazor Interactive Server aparece en la vida de los servicios `Scoped`: su alcance no debe equipararse automáticamente con una petición HTTP MVC, porque los circuitos interactivos tienen una duración propia. Nuestro servicio no guarda modificaciones entre llamadas, de modo que la práctica no necesita explorar esas implicaciones operativas.

Prerenderizado y dobles inicializaciones

En una Blazor Web App es posible representar una página inicialmente en el servidor y luego crear la instancia interactiva asociada al navegador. Con prerenderizado habilitado, `OnInitializedAsync` puede ejecutarse una vez durante el prerenderizado y otra durante la inicialización interactiva. En una carga de datos costosa, esta repetición puede resultar relevante. Si el método no fuera idempotente, incluso podría generar resultados indeseados.

El laboratorio desactiva el prerenderizado de P3.6 con `@rendermode @(new InteractiveServerRenderMode(prerender: false))`. Queremos observar una sola secuencia de carga que evoluciona visiblemente desde «Cargando» hasta «Lista

preparada». Debemos explicar, sin embargo, que el prerenderizado por defecto puede mejorar la experiencia de carga inicial y que desactivarlo no es un estándar universal. En aplicaciones mayores existen técnicas para conservar estado de prerenderizado y evitar operaciones duplicadas, fuera del alcance de este módulo.

Cuatro estados conceptualmente diferentes

Consideremos una pantalla con estas posibilidades: todavía no se ha ejecutado la carga, la operación está en curso, la operación ha terminado con datos y la operación ha terminado sin datos. Estos estados no significan lo mismo. En nuestro proyecto inicial, `estado = "Servicio pendiente de activar"` indica que el método aún está comentado. Al activarlo, aparece `"Cargando productos..."` antes de completar `ObtenerAsync`. Cuando termina, `"Lista preparada"` anuncia la finalización. Si la lista está vacía, el reto muestra una frase adicional, no un error técnico.

El alumno debe distinguir también un quinto estado que estudiará en M04: **la operación falló**. Un servicio HTTP puede devolver un código de error o no ser alcanzable; eso necesita una representación y manejo específicos. M03 utiliza un servicio controlado para no mezclar fallos de red con la comprensión de los eventos y el ciclo de vida.

Diagnóstico de servicio no registrado

Si eliminásemos temporalmente el registro de `IProductosMemoriaService` en `Program.cs` y dejáramos el `@inject` de la página, el contenedor no sabría cómo suministrar la dependencia. No sería un problema de routing ni del CSS de las tarjetas. La causa estaría en la configuración de servicios. Por eso el primer paso de P3.6 pide abrir y comprender tanto la interfaz como su registro antes de activar `OnInitializedAsync`.

Otro fallo diferente sería registrar correctamente el servicio pero olvidar activar el método de inicialización. En ese caso el proyecto puede compilar y la página representar el mensaje inicial de «Servicio pendiente de activar» para siempre. La depuración correcta consiste en comprobar que el bloque de

la práctica está activo y que se está ejecutando la versión correspondiente del proyecto, no en cambiar la interfaz de DI.

Diagnóstico de operación completada sin actualización visual

Supongamos que `productos = await Catalogo.ObtenerAsync()` se ejecuta correctamente, pero el código omite `cargando = false`. La lista contiene tres objetos, pero la condición del Razor sigue siendo falsa. El alumno podría afirmar que el servicio «no devolvió nada» porque no ve tarjetas. La realidad sería otra: **el estado de presentación impide mostrarlas**. El método necesita mantener coherentes tanto la colección como la señal de finalización.

Este caso demuestra la utilidad de representar explícitamente el estado de carga. En aplicaciones complejas, se usan modelos de estado más expresivos; aquí bastan una bandera y un texto para mostrar la idea. No introducimos una biblioteca de gestión de estado para resolver un problema que el alumno ya puede comprender con dos campos.

Un resultado vacío no es necesariamente un resultado incorrecto

Cuando el servicio devuelve `Array.Empty<Producto>()`, ha completado la operación con una colección válida pero sin elementos. La condición `productos.Count > 0` impide representar tarjetas, y el reto introduce un mensaje para explicar lo ocurrido. Es distinto de una solicitud de red fallida y distinto de un servicio todavía pendiente. La observación positiva del reto no es «ver tres productos», sino «ver que el componente terminó correctamente y explica que no hay productos».

Este matiz es importante porque preparará el tratamiento de resultados y errores HTTP de M04. Aprenderemos entonces que una respuesta satisfactoria con colección vacía, una respuesta 404 y una respuesta de error de servidor pueden merecer representaciones distintas. Aquí el alumno aprende primero a no confundir **cero elementos** con **espera infinita**.

Integración de toda la jornada en una sola historia

Imagina que el usuario visita la página de productos de la futura aplicación B. El sistema debe localizar la ruta de componente, representar el layout,

obtener datos de un servicio y crear tarjetas. El usuario puede interactuar con esa representación, modificar una selección o solicitar una acción en una tarjeta. Cada parte de esa historia procede de un punto del módulo: routing en 3.2, composición en 3.3, eventos en 3.4, callbacks en 3.5 e inicialización asíncrona en 3.6.

No hemos conectado todavía todos esos mecanismos en un CRUD remoto completo; hacerlo sin haber aprendido las piezas separadas haría más difícil atribuir cada comportamiento a su causa. La progresión del módulo está diseñada para que el alumno pueda explicar cada eslabón antes de combinarlos con HTTP en M04.

Autoevaluación razonada

Describe qué archivo revisas primero ante las siguientes incidencias: una página devuelve 404; una tarjeta muestra un precio incorrecto; un botón no desencadena ningún evento; un hijo emite un Id distinto del esperado; la colección de productos se carga pero nunca se representa; la pantalla informa de lista vacía después de una carga correcta. Las respuestas deberían relacionar síntomas con routing, parámetro recibido, renderizado interactivo, EventCallback, bandera de carga y estado vacío respectivamente.

Si todas tus respuestas consisten en «cambiar Program.cs» o «reconstruir la aplicación», todavía no estás usando la separación de responsabilidades que hemos practicado. El resultado formativo de M03 no es la cantidad de componentes escritos, sino la capacidad de comprender cómo cooperan para construir una interfaz interactiva reproducible.

Estudio aplicado I — Seguir una página Blazor de extremo a extremo

Para comprender un framework de interfaz es útil analizar una petición real sin saltar directamente al fragmento más vistoso. Tomemos `/productos`, la dirección de la aplicación B que aparece desde P3.2. En el laboratorio, el navegador solicita esa ruta al servidor ASP.NET Core. El proyecto está configurado en `Program.cs` para mapear componentes Razor. El router puede localizar `Productos.razor` porque declara `@page "/productos"`. La página se representa

dentro del layout y utiliza el modelo de datos que corresponde al estado de práctica. Esta cadena explica la aparición inicial del catálogo sin atribuir al navegador capacidades que realmente pertenecen al servidor.

Primera frontera: HTTP y el endpoint de componentes

En MVC, una URL podía seleccionar una acción de un controlador que devolvía `View()`. En Blazor Web App, el endpoint de componentes permite atender la ruta cuya página declara `@page`. No es necesario que el alumno cree una acción `ProductosController.Index` para llegar a `Productos.razor`. El hecho de que ambos sistemas usen ASP.NET Core permite reutilizar conocimientos del host, pero no convierte los componentes en acciones MVC.

La diferencia se aprecia al diagnosticar una dirección no encontrada. Si solicitamos `/productos-inexistentes`, el router no tiene una página registrada con ese nombre. El contenido de `NotFound` ofrece una salida. Cambiar el precio de Cuaderno en `Producto.cs` no solucionará el fallo, porque se produce antes de representar la colección. Este ejemplo ayuda a distinguir **selección de endpoint** de **obtención y representación de datos**.

Segunda frontera: el árbol de componentes

Una vez identificada `Productos.razor`, el sistema representa un árbol: la estructura inicial de `App.razor`, la selección de página de `Routes.razor`, el layout común y el componente específico. Desde P3.3, también incluye varias instancias de `TarjetaProducto`. Cada nivel tiene responsabilidades distintas. El componente raíz proporciona el documento y los recursos; el router selecciona; el layout organiza la navegación; la página posee la lista; la tarjeta representa un producto.

Esta organización permite razonar sobre un error sin rehacer toda la aplicación. Si el menú no muestra Productos, revisamos `MainLayout.razor`. Si la ruta no existe, revisamos `@page` y el router. Si los nombres aparecen cambiados, revisamos la colección y el parámetro. Si todos los artículos comparten un Id, revisamos el valor que el padre pasa al hijo. Si el estilo falta, comprobamos la hoja `wwwroot/app.css`. Cada síntoma pertenece potencialmente a un nivel diferente del árbol.

Tercera frontera: datos C# y representación HTML

En la versión de P3.3 la lista está escrita directamente en el componente y cada iteración de `@foreach` produce una tarjeta. La expresión `@Producto.Nombre` del hijo no crea automáticamente otro objeto: lee una propiedad del objeto recibido. Tampoco el atributo HTML `data-producto-id` realiza un enlace de datos por sí mismo; sirve como representación visible de una identidad que ya pertenecía al modelo.

Por ello, al inspeccionar el HTML, el alumno puede encontrar tres artículos que contienen texto y valores, mientras que en el archivo Razor observa un único bloque `TarjetaProducto` dentro de un bucle. El modelo de componentes convierte una descripción repetible en varias instancias de representación. La utilidad principal no está en escribir menos líneas por sí mismo, sino en conservar un contrato único que permita modificar el marcado de todas las tarjetas.

Cuarta frontera: interacción y estado

En P3.4 aparece un botón que cambia `seleccion`. A diferencia de la carga inicial del documento, este evento puede actualizar una pequeña parte de la interfaz sin navegar a otra URL. La representación depende del campo del componente y del proceso de actualización que coordina Blazor. Una nueva ruta no es necesaria para que aparezca «Seleccionado: Lápiz»; esa frase procede del estado local modificado por `Seleccionar(int)`.

No debemos convertir esta característica en una generalización excesiva. Un componente interactivo puede navegar, llamar a servicios y provocar solicitudes HTTP cuando su función lo requiere. En nuestro ejercicio concreto, la selección no necesita esas operaciones. Aprendemos a elegir el mecanismo más simple que explique el comportamiento deseado.

Quinta frontera: componente hijo y notificación

P3.5 mueve el botón a TarjetaProducto. La tarjeta ahora conoce su propia intención de seleccionar, pero no es dueña del campo `seleccion` del padre. Utiliza `EventCallback<int>` para comunicar el Id. La selección aparece después de que el padre reciba la notificación y modifique su estado. Este diseño delimita

una frontera dentro del árbol de componentes, tan relevante como la frontera HTTP inicial.

El alumno puede seguir el evento con puntos de interrupción: método del hijo, `InvokeAsync`, método del padre, asignación del campo y nueva representación. La presencia de un callback tipado hace visible esa relación. No debe confundirse con un servidor de mensajes general ni con una API REST entre dos componentes del mismo árbol.

Sexta frontera: adquisición asíncrona

En P3.6 los datos dejan de crearse dentro del componente y proceden de `IProductosMemoriaService`. La página solicita el servicio a través de `@inject` y espera el resultado en `OnInitializedAsync`. La frontera ya no es un dato constante local, sino una dependencia proporcionada por el contenedor. Esta evolución prepara M04, donde ese servicio puede relacionarse con una API remota.

En el estado actual, el servicio sigue siendo controlado y local. Eso permite concentrarse en la transición de carga sin confundir fallos de red, HTTP o JSON con el funcionamiento de componentes. El marcado distingue «esperando», «completado con datos» y «completado sin datos» si se activa el reto correspondiente. Cada estado es una parte de la interfaz y necesita una representación comprensible.

Qué aprende el alumno al reconstruir el recorrido

La cadena completa no es «Blazor hace todo». Es «ASP.NET Core sirve un endpoint de componentes; el router identifica la página; el layout la compone; la página obtiene o construye datos; los hijos reciben parámetros; los eventos modifican estado; el framework vuelve a representar lo necesario». Esta explicación proporciona una base para investigar comportamientos inesperados sin recurrir a modificaciones arbitrarias.

Una buena autoevaluación consiste en señalar un archivo para cada eslabón y justificar qué ocurriría si esa pieza estuviera ausente o mal configurada. Si falta una ruta, fallará la selección del componente; si falta un parámetro, la tarjeta no recibirá su contrato; si falta un callback, el hijo no podrá notificar al padre; si falta el registro del servicio, la inyección puede fallar; si falta el

modo interactivo, el botón no ejecutará el evento que esperamos. Son causas distintas con síntomas que se pueden observar por separado.

Estudio aplicado II — Elegir una estructura para cada necesidad

Una formación profesional no debería presentar MVC, Razor Pages y Blazor como tres nombres intercambiables o como versiones sucesivas de una misma tecnología. Son formas de organizar la aplicación y su interfaz con diferencias reales. Hemos elegido la aplicación MVC A para formularios y persistencia, y la aplicación Blazor B para componentes interactivos. Esta elección tiene una razón didáctica que también puede informar decisiones profesionales.

Una página de consulta con navegación tradicional

Imagina una página de catálogo que muestra información principalmente de lectura, con enlaces a detalle y cambios ocasionales mediante formularios. MVC puede ser un enfoque razonable: el controlador recibe la solicitud, obtiene el modelo y elige una vista. Razor Pages también puede ser adecuado si la funcionalidad se organiza naturalmente alrededor de pantallas relativamente autónomas con handlers GET y POST. No necesitamos un circuito Interactive Server para cada simple enlace.

Eso no significa que las soluciones tradicionales sean «estáticas» en sentido de datos. MVC puede generar información actualizada en cada petición y Razor Pages puede validar formularios y utilizar EF Core. Debemos distinguir entre contenido que cambia **entre solicitudes** y una experiencia de componente que reacciona a eventos **dentro de una sesión interactiva**.

Una interfaz que depende de numerosos estados visuales

Imagina ahora una pantalla que necesita seleccionar productos, activar filtros, editar campos y actualizar inmediatamente partes de la representación. Una aplicación basada en componentes puede permitir expresar esas relaciones directamente en C#, con parámetros, estado y eventos. Blazor ofrece herramientas para construir esa experiencia. El valor no está en que un clic sea técnicamente imposible con MVC, sino en que el diseño y la composición de eventos pueden resultar más naturales en un modelo interactivo.

No podemos evaluar este diseño atendiendo solo a la cantidad de líneas. Interactive Server también implica dependencia de conexión, circuitos y recursos de servidor. Es posible que una interfaz muy pequeña no necesite esa complejidad. La decisión debe considerar el comportamiento de usuario, las restricciones del entorno, las capacidades del equipo y los requisitos de operación.

La relación con la infraestructura

En M01 y M02 construimos una aplicación MVC que, para el CRUD persistente, utiliza LocalDB. En M03 la aplicación B funciona sin base de datos ni API, porque el objetivo es dominar componentes. La ausencia de almacenamiento no hace que Blazor sea incompatible con EF Core; solo significa que hemos decidido no mezclar conceptos antes de tiempo.

En M04 añadiremos comunicación HTTP a B. Esto permitirá observar que la interfaz interactiva y el backend pueden estar separados. En M05 la consolidación final recaerá en B, mientras que las demostraciones introductorias de despliegue utilizarán la aplicación MVC A para evitar infraestructura innecesaria. El diseño del curso responde a una progresión de aprendizaje, no a una regla según la cual cada tecnología solo pueda realizar las tareas que hacemos en ella.

Un caso en el que Razor Pages sigue siendo razonable

Una pantalla administrativa pequeña que presenta un formulario y procesa un POST podría utilizar Razor Pages de manera clara. El `PageModel` concentra los handlers y la vista muestra los valores y errores. Si el usuario realiza unos pocos envíos de formulario, ese diseño puede ser simple y mantenible. No debemos introducir por obligación un árbol de componentes con estado interactivo si no aporta al requisito.

En P2.5 hicimos una demostración breve de esta opción. No construimos un segundo CRUD ni otra aplicación. Esa decisión preservó el tiempo para aprender Blazor en la jornada siguiente y muestra que una formación bien diseñada puede enseñar varias alternativas sin afirmar que todas necesitan la misma profundidad.

Un caso en el que la reutilización es especialmente valiosa

Si la pantalla de productos aparece en varias secciones, un componente TarjetaProducto ayuda a conservar un contrato de representación uniforme. Si cambia el modo de mostrar el precio, la tarjeta se modifica una vez. Si cambia el origen de datos, la tarjeta puede mantenerse igual siempre que reciba un objeto compatible. Si necesita informar de un clic al padre, dispone de un callback explícito.

En una vista MVC también podemos usar parciales y componentes de vista, pero sus mecanismos se integran con el ciclo de renderizado del servidor. La comparación relevante no es que uno «permita reutilizar» y otro no, sino cómo se combinan reutilización, navegación e interacción en cada arquitectura.

Un caso en el que el estado local no es suficiente

Imagina que el alumno añade Regla en P3.4 y, después de recargar, espera que el producto aparezca en otro ordenador. Nuestro componente no puede cumplir ese requisito, porque solo modifica una lista local de instancia. No deberíamos «arreglar» el problema con una variable `static` global que comparta datos accidentalmente entre usuarios. Necesitaríamos diseñar un origen de datos adecuado, una API o un almacenamiento persistente según el requisito real.

La conclusión es que un estado útil para la interfaz no siempre coincide con el estado de negocio que debe persistir. Comprender esa diferencia será esencial en M04 y M05. M03 enseña el flujo de componentes; no promete permanencia de todos los cambios.

Una plantilla de decisión razonada

Al elegir entre MVC, Razor Pages y Blazor, formula cuatro preguntas. ¿Cuál es la experiencia de interacción requerida? ¿Qué partes se pueden reutilizar? ¿Dónde deben residir el estado y los datos? ¿Qué dependencias operativas podemos aceptar? En el curso, esas respuestas están controladas para facilitar aprendizaje: A tiene formularios y persistencia; B tiene eventos y componentes; una API pequeña se suministra cuando corresponda.

La elección de una tecnología no debe justificarse porque sea «más moderna». Una justificación sólida relaciona capacidades con requisitos concretos y reconoce también sus costes. Ese criterio es tan importante como saber escribir una directiva Razor.

Estudio aplicado III — Calidad de los componentes sin complejidad innecesaria

La calidad de una interfaz no se limita a que compile y muestre tarjetas bonitas. Un componente debería expresar una responsabilidad clara, presentar datos correctos, reaccionar de forma predecible y permitir comprender un estado de error o una ausencia de contenido. M03 ofrece ejemplos pequeños que ayudan a formar este criterio sin desarrollar una arquitectura empresarial.

Contratos de entrada claros

TarjetaProducto recibe un objeto de tipo `Producto`. El parámetro se declara públicamente y su uso aparece en el mismo archivo. Esta visibilidad ayuda a comprender de dónde proceden los datos. Un componente que dependiera de muchas variables globales ocultas sería más difícil de reutilizar y verificar. No estamos obligados a usar siempre un único objeto, pero conviene que cada parámetro forme parte de un contrato comprensible.

También es importante respetar qué información es realmente necesaria. La tarjeta no necesita una cadena de conexión de SQL Server, una instancia de DbContext ni la URL de una API para dibujar un nombre y un precio. Introducir dependencias que no responden a su responsabilidad complicaría el componente y su utilización posterior.

Estados visibles y lenguaje comprensible

En P3.6 la página puede mostrar «Cargando productos...» y «Lista preparada». Estos textos explican al usuario en qué situación se encuentra. El reto añade un mensaje para lista vacía. Una interfaz que solo desaparece mientras espera obliga al usuario a adivinar si la operación continúa, terminó sin datos o ha fallado.

El texto de estado forma parte de la experiencia, no es un log de desarrollo. En aplicaciones reales necesitaremos considerar accesibilidad, mensajes de error y recuperación, pero el principio ya puede verse en un ejemplo pequeño: distintos estados del componente merecen representaciones distinguibles. No es necesario construir un sistema completo de notificaciones para comprobarlo.

Pruebas positivas y negativas

El contador de P3.1 necesita una prueba real de clic; descargar HTML no basta. El catálogo de P3.2 necesita comprobar la ruta y el número de filas. TarjetaProducto de P3.3 necesita comprobar identidades y contenidos diferentes. P3.4 necesita aceptar una entrada válida y rechazar otra vacía sin añadir producto. P3.5 necesita comprobar la notificación y el efecto sobre el padre. P3.6 necesita mostrar carga completada y lista vacía. Estos criterios son específicos de cada responsabilidad.

Una prueba que solo verifica que existen archivos o cadenas en el repositorio puede ser útil para detectar problemas estructurales, pero no demuestra que un usuario pueda completar la interacción. Por eso el material práctico insiste en abrir las páginas y observar sus cambios en un navegador. El propósito no es convertir al alumnado en especialista en automatización de pruebas, sino enseñar a distinguir una expectativa funcional de una afirmación no comprobada.

Diagnósticos que no destruyen el estado conocido

Los proyectos del curso conservan bloques de activación y estados iniciales conocidos. Esto permite reproducir las transiciones sin hacer cambios irreversibles. Por ejemplo, si un evento deja de funcionar después de activar un botón, podemos comprobar su método y la directiva de renderizado. Si la lista aparece duplicada, podemos revisar las variantes de componentes activas. Si la carga no termina, podemos revisar la bandera `cargando`.

Un diagnóstico disciplinado no consiste en borrar el proyecto y volver a generarlo cada vez que aparece un síntoma. Primero identifica la última transición realizada, el archivo cambiado y el resultado observado. Después

comprueba la causa que esa transición podría haber introducido. Este método sirve para proyectos de aprendizaje y también para aplicaciones profesionales.

Accesibilidad y semántica sin convertirla en otro módulo

Los ejemplos utilizan encabezados, botones y etiquetas HTML reconocibles. Un botón real comunica una acción, mientras un enlace comunica navegación. Elegir elementos semánticos apropiados ayuda tanto a la accesibilidad como a la comprensión del código. No es necesario desarrollar aquí un curso completo de accesibilidad web, pero sí podemos evitar patrones confusos como usar un enlace vacío para simular una acción que no navega.

El componente `TarjetaProducto` utiliza un artículo con encabezado y contenido. La página dispone de un título principal. El layout ofrece navegación común. Los mensajes de carga aparecen como texto. Son decisiones sencillas que producen una interfaz más comprensible y facilitan las pruebas de navegador, además de su utilidad para el usuario.

Cierre didáctico

Una formación de Blazor para quienes ya conocen C# debe invertir tiempo en explicar el modelo de componentes, no en repetir cada sintaxis del lenguaje. El alumno puede leer una lambda o un record, pero necesita entender cómo esa expresión se relaciona con una ruta, un parámetro o un evento. La calidad del material depende de la continuidad entre esas ideas y de que exista un proyecto real que permita observarlas.

Al finalizar, el alumno debería poder defender por qué P3.6 usa un servicio en memoria, por qué P3.5 usa un callback y por qué P3.1 compara dos modos de renderizado. Las tres respuestas deben referirse a responsabilidades y comportamientos concretos. Esta capacidad de explicación es más valiosa que memorizar un conjunto de etiquetas y constituye la preparación necesaria para integrar APIs y seguridad básica en M04.

Bibliografía y documentación de referencia del módulo

Los ejemplos se han diseñado para **ASP.NET Core 8**, con la terminología de Blazor Web App e Interactive Server propia de esa versión. La documentación actual de Microsoft puede presentar otras versiones seleccionadas por defecto; al consultar un enlace debe mantenerse el parámetro `view=aspnetcore-8.0` para no mezclar fragmentos de versiones distintas.

- Microsoft Learn: Visión general de Blazor.
- Microsoft Learn: Modos de renderizado.
- Microsoft Learn: Componentes Razor.
- Microsoft Learn: Eventos en componentes.
- Microsoft Learn: Enlace de datos.
- Microsoft Learn: Ciclo de vida.
- Microsoft Learn: Navegación y routing.